

Capacitación en Ciencia de Datos e Inteligencia Artificial Parte II

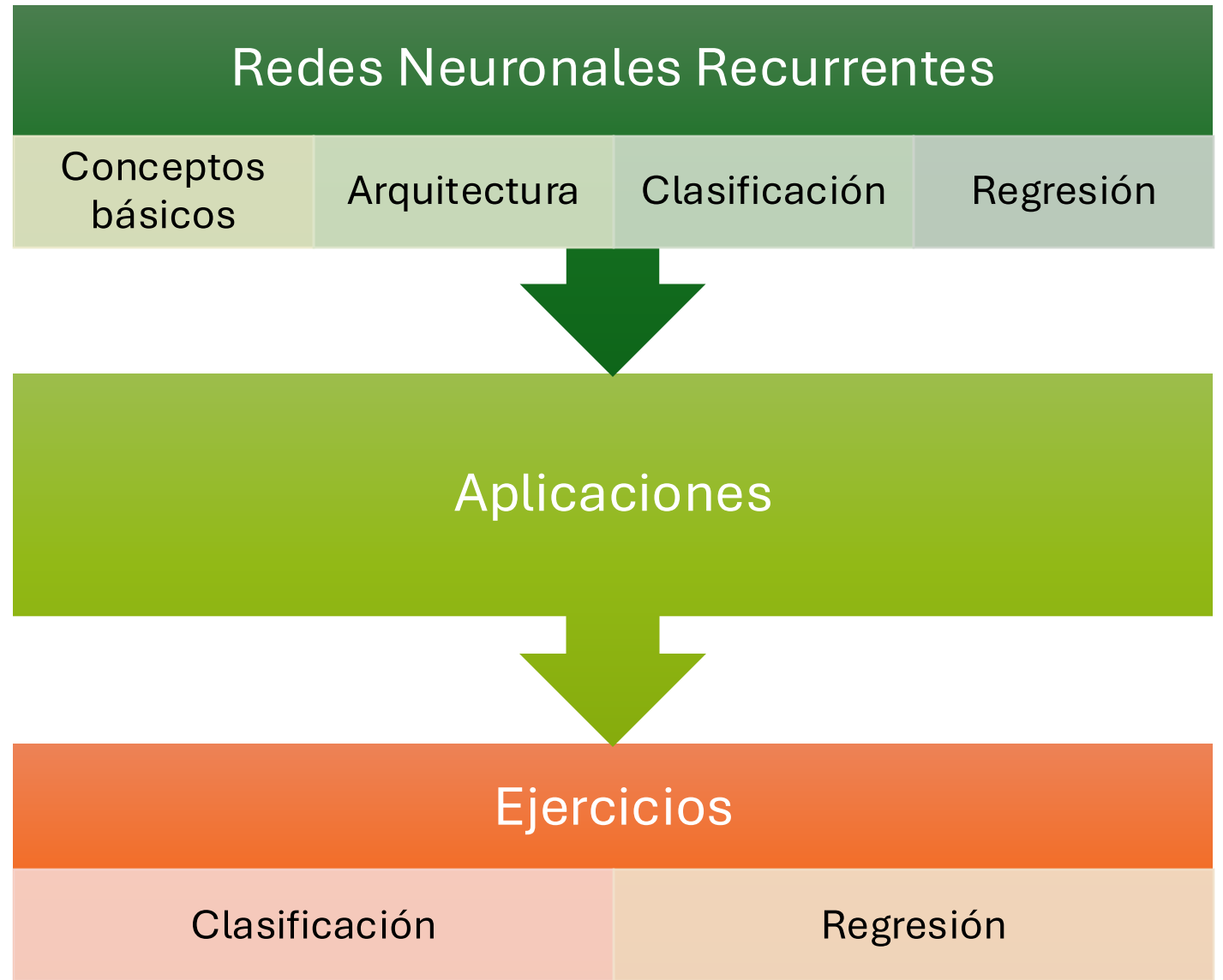
Sesiones 7 y 8: Redes
Neuronales Recurrentes

Presenta:

Dr. Ulises Olivares Pinto

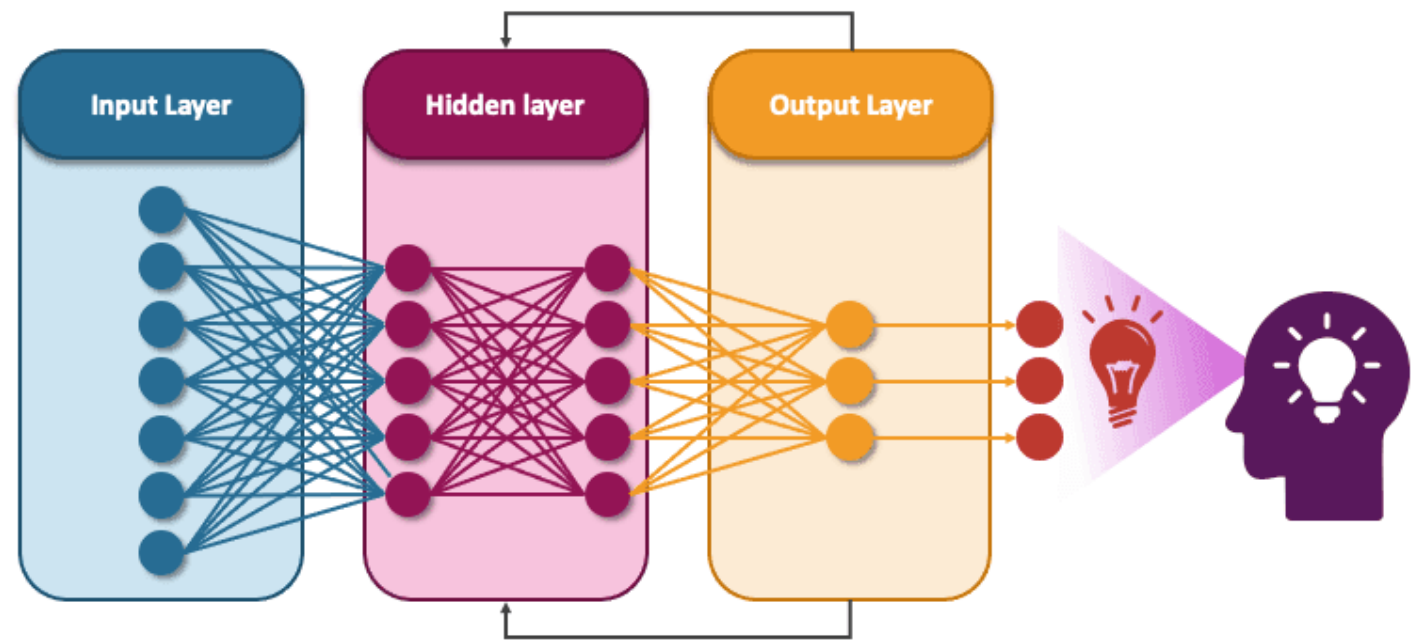


Contenido

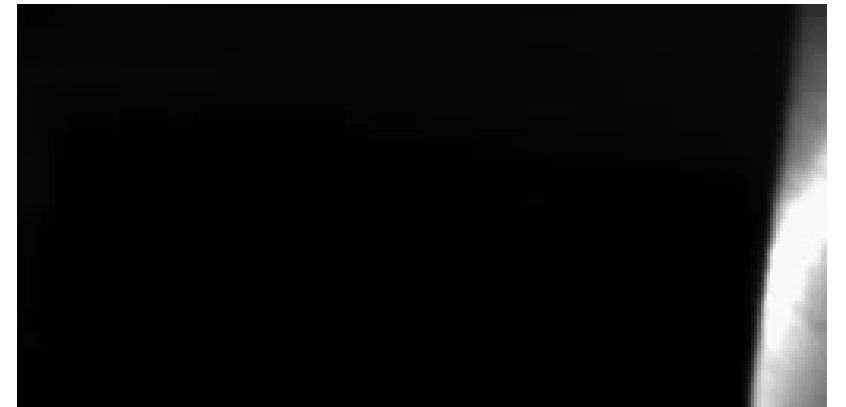


Redes Neuronaes Recurrentes

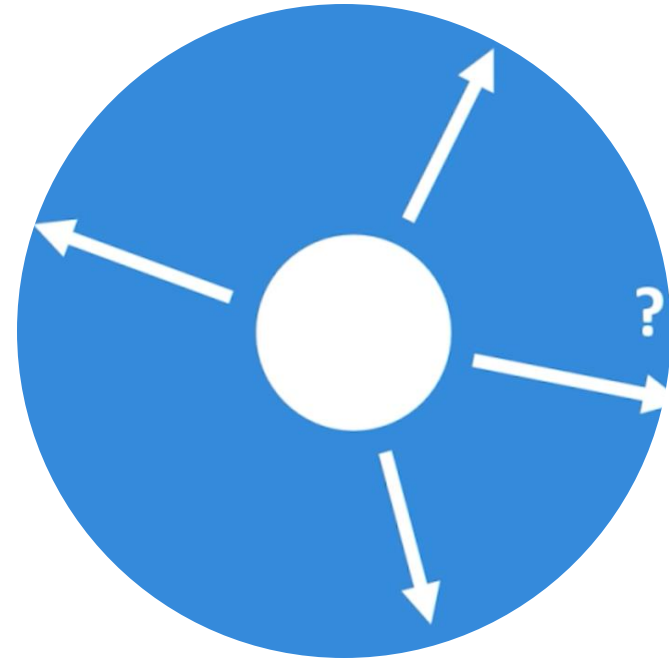
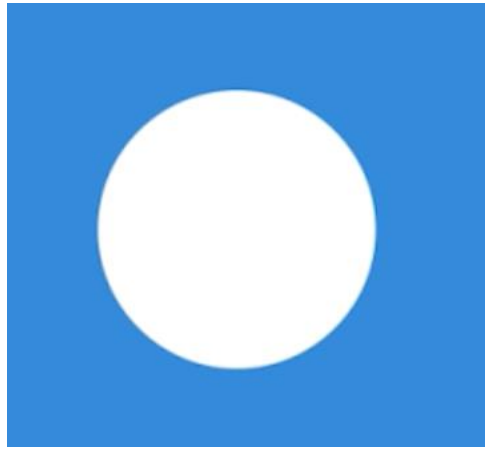
Deep Learning



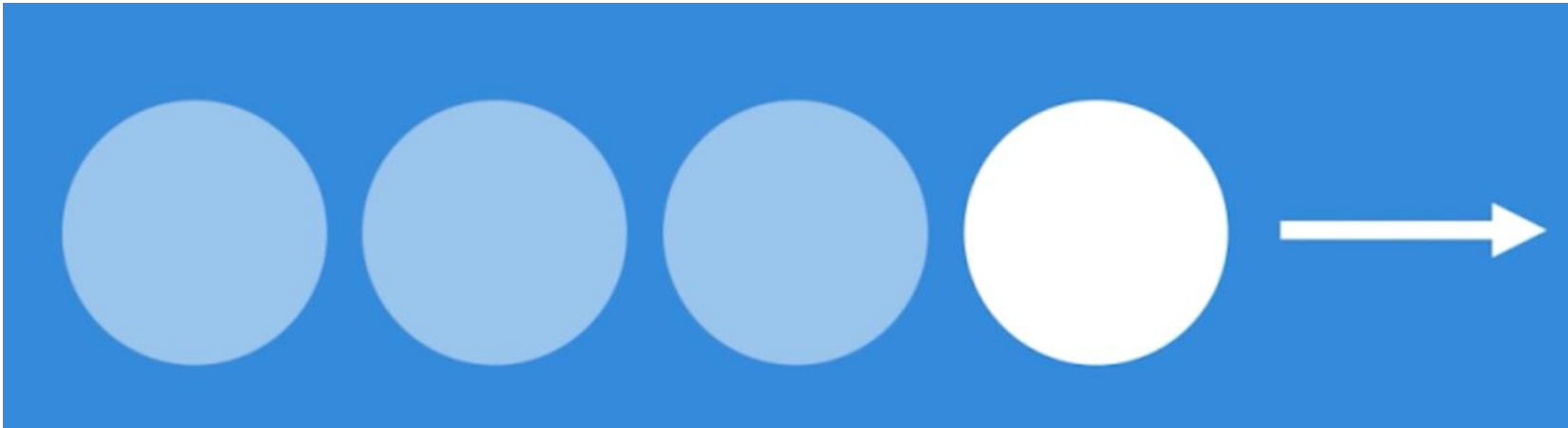




Dada la siguiente imagen
¿Es posible predecir hacia donde se moverá?



Dada la siguiente imagen
¿Es posible predecir hacia donde se moverá?



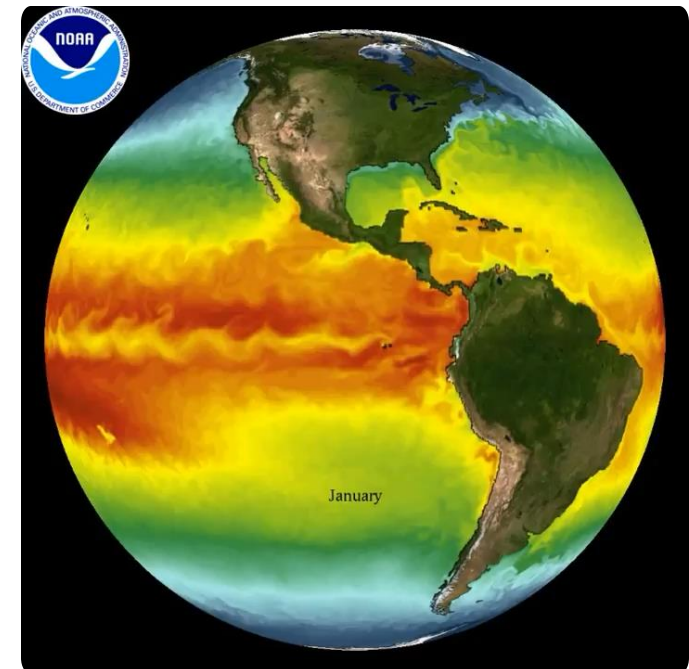
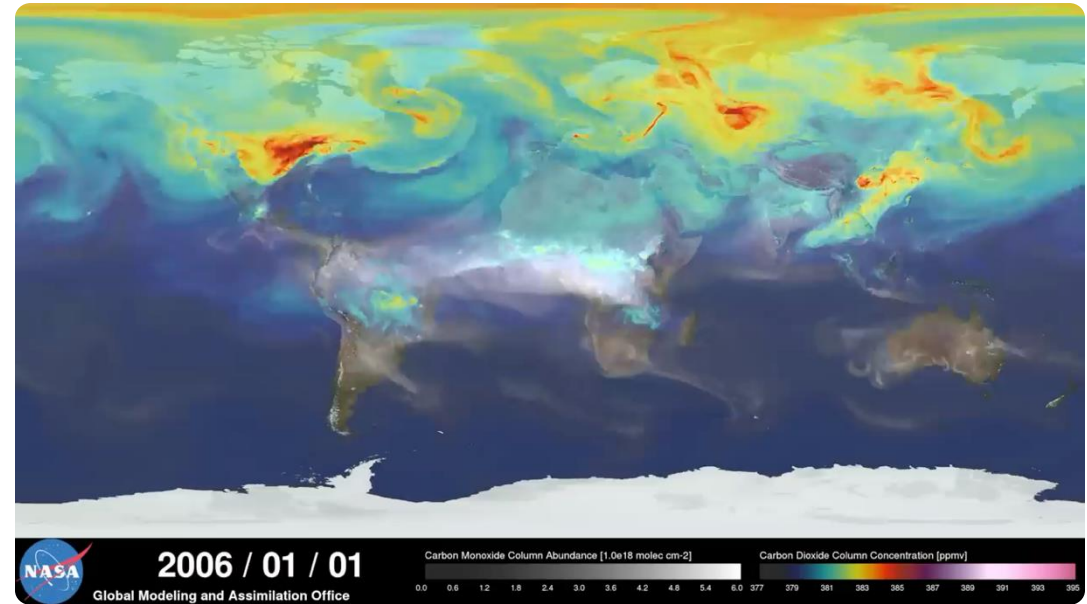
Datos secuenciales



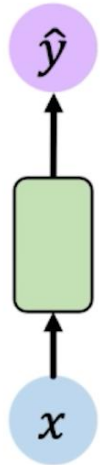
08, 2019

Secuencias en el mundo real

Secuencias en la vida real



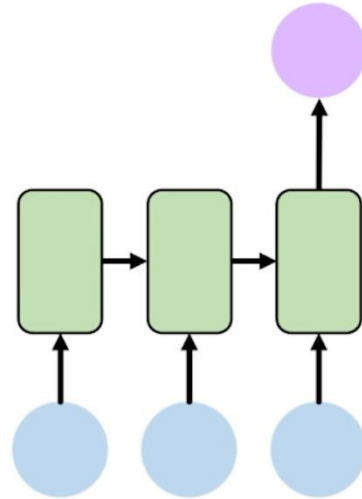
Datos Secuenciales



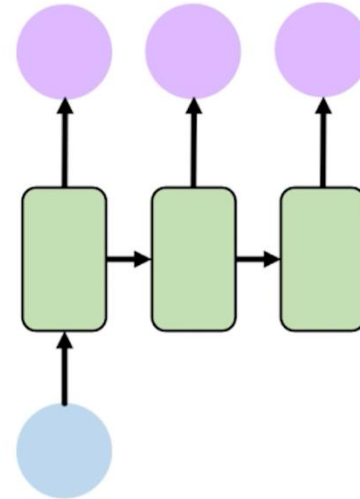
One to One
Binary Classification



"Will I pass this class?"
Student → Pass?



Many to One
Sentiment Classification

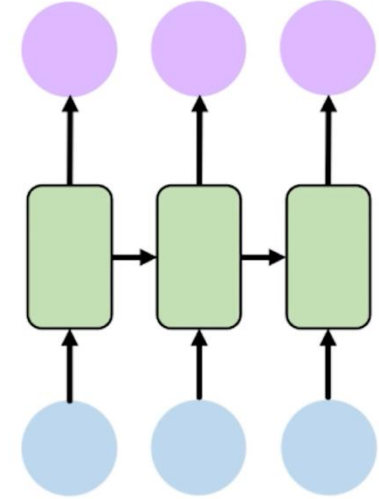


One to Many
Text Generation
Image Captioning

Image Captioning



"A baseball player throws a ball."

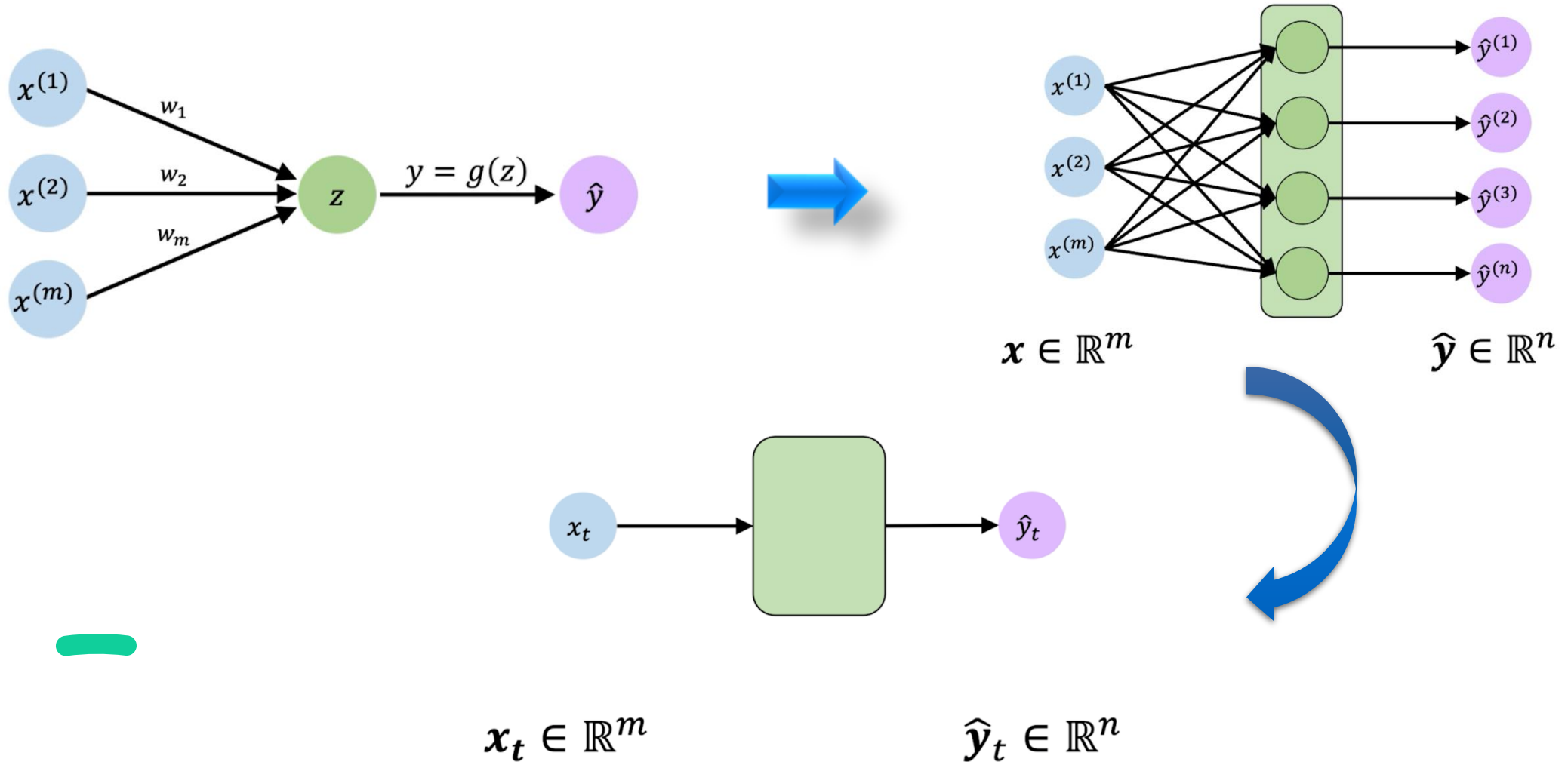


Many to Many
Translation & Forecasting
Music Generation

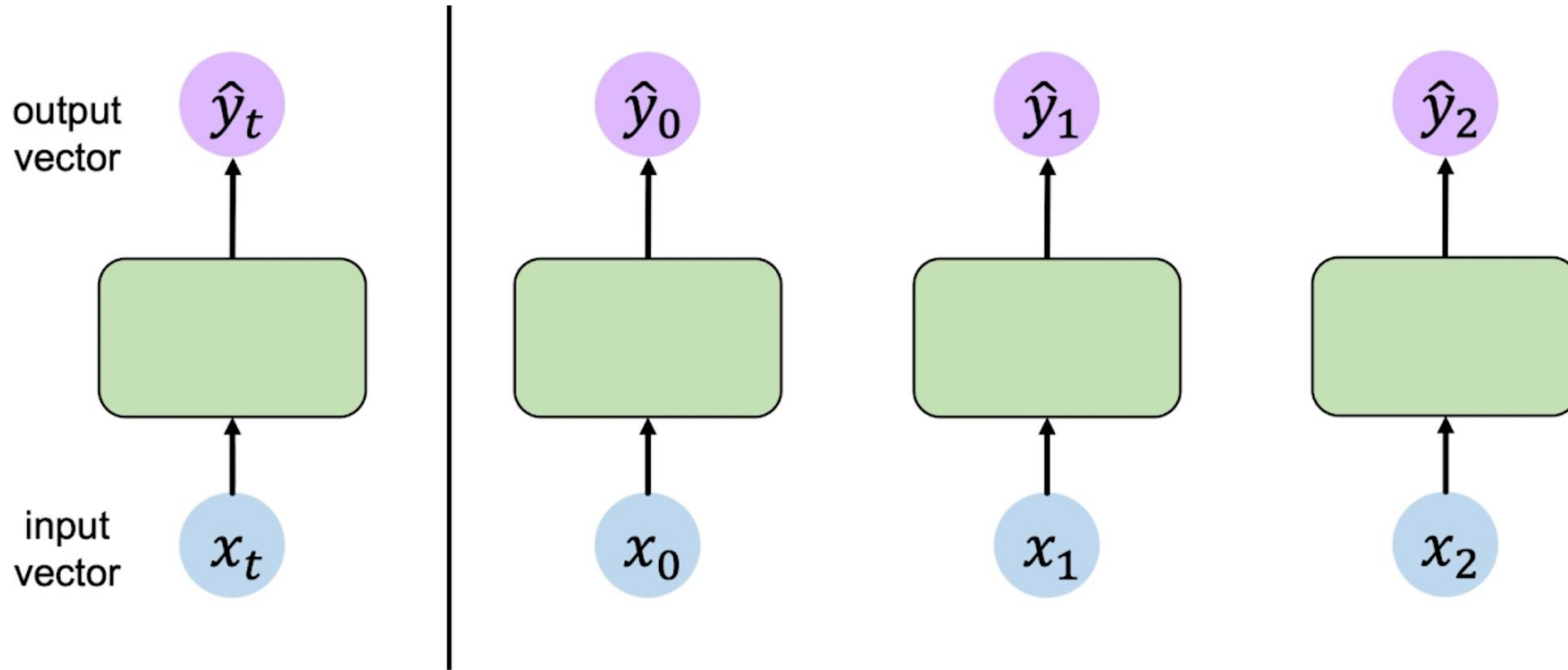
Machine Translation



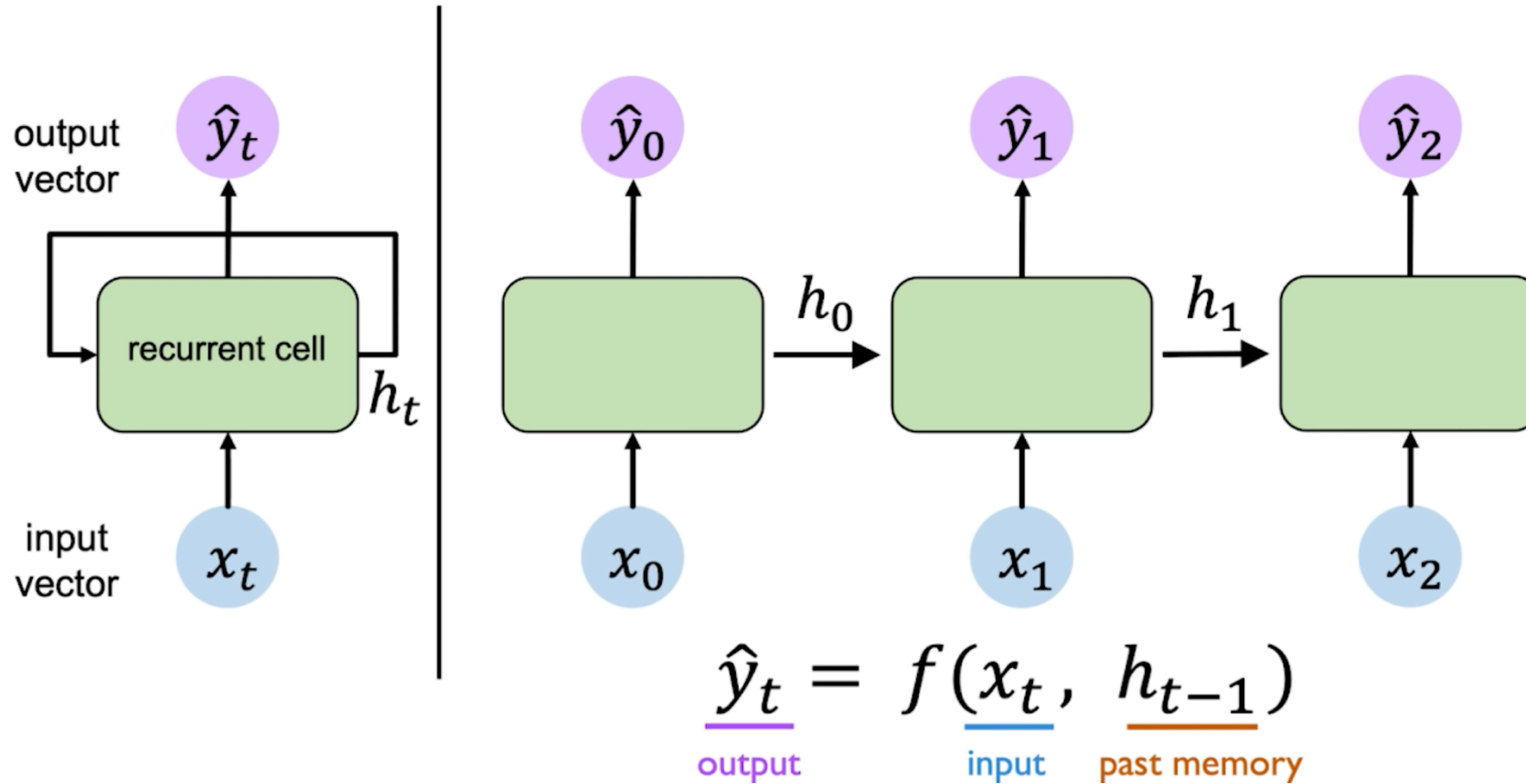
Redes Neuronales (Feed-forward)



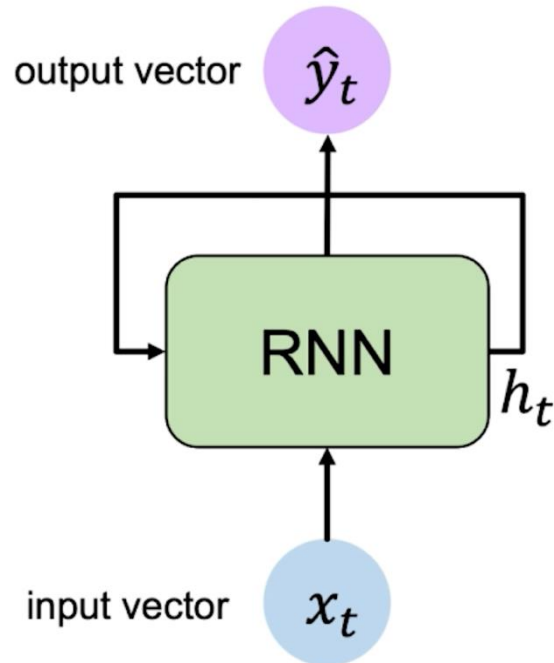
Neuronas con recurrencia



Neuronas con recurrencia (RNNs)



Neuronas con recurrencia (RNNs)



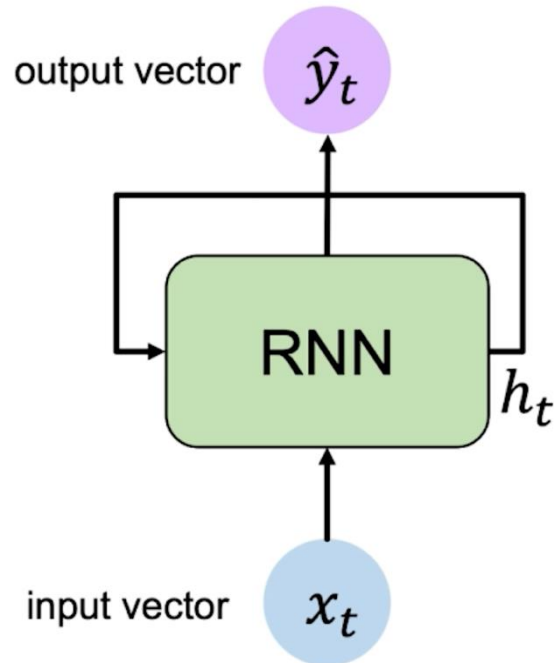
Apply a **recurrence relation** at every time step to process a sequence:

$$\boxed{h_t} = \boxed{f_W}(\boxed{x_t}, \boxed{h_{t-1}})$$

cell state function with weights W input old state

Las RNNs tienen un estado h_t , el cual se actualiza en cada iteración cuando la secuencia es procesada

Neuronas con recurrencia (RNNs)



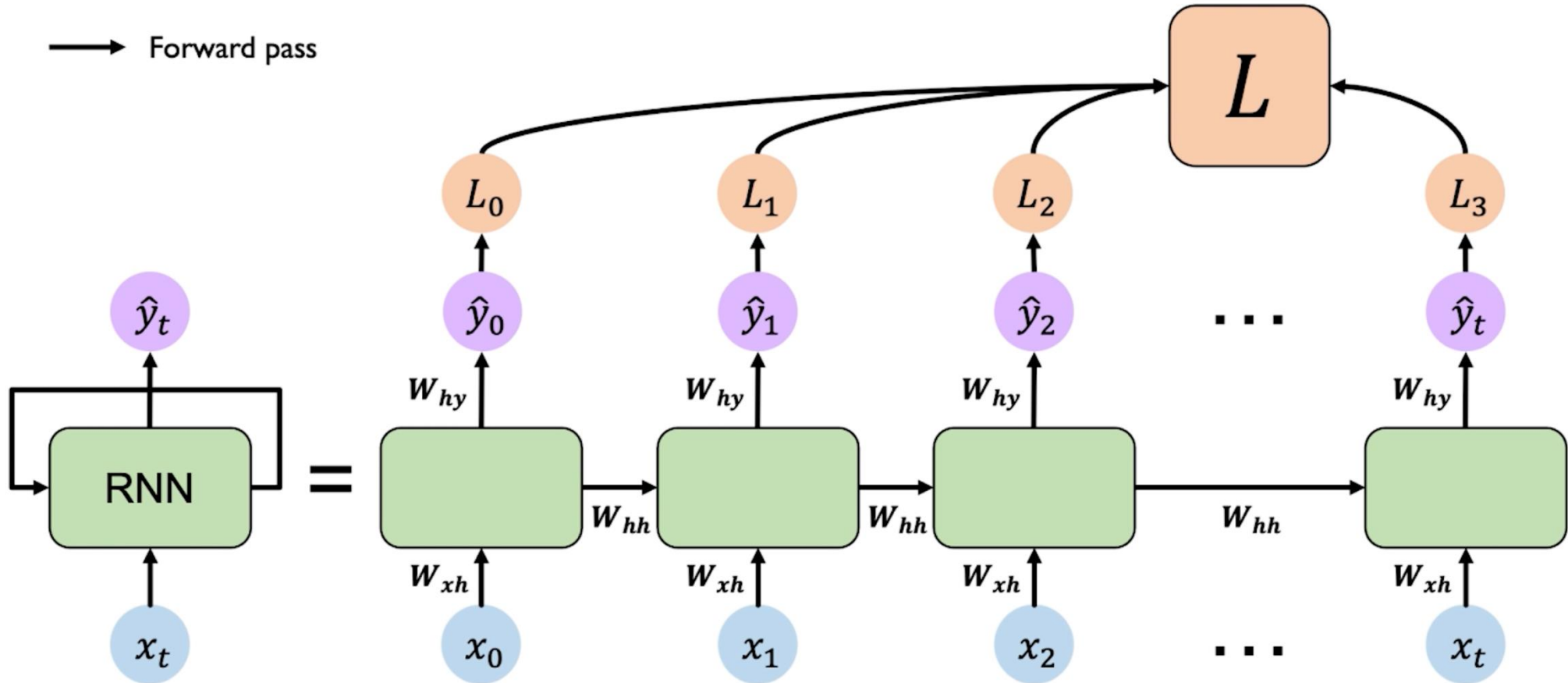
Apply a **recurrence relation** at every time step to process a sequence:

$$\boxed{h_t} = \boxed{f_W}(\boxed{x_t}, \boxed{h_{t-1}})$$

cell state function with weights W input old state

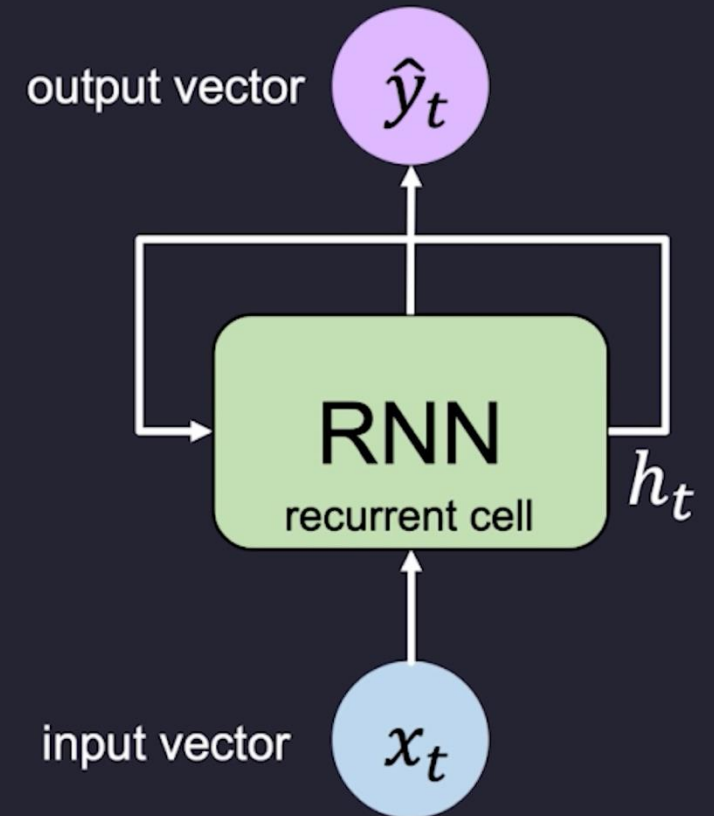
Las RNNs tienen un estado h_t , el cual se actualiza en cada iteración cuando la secuencia es procesada

RNNs: A lo largo del tiempo

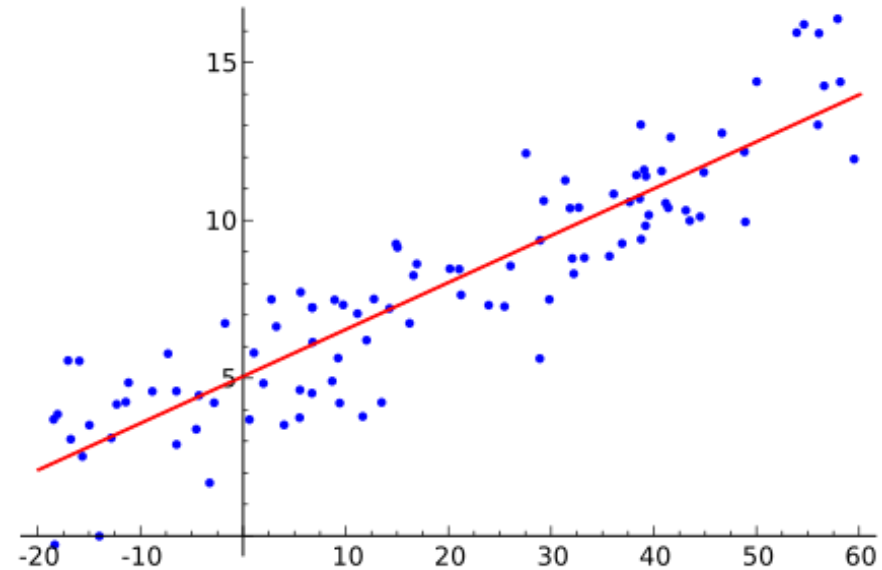


RNNs en Tensor Flow

```
tf.keras.layers.SimpleRNN(rnn_units)
```



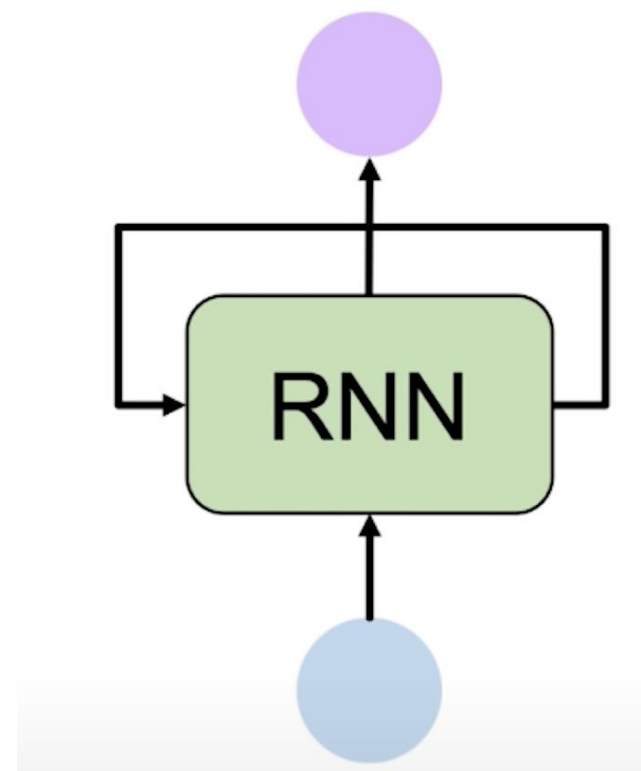
Ejemplo:
Predicción de
siguiente
número



RNNs para el modelado secuencial

- Criterios de diseño:

1. Tener en cuenta la longitud de las secuencias
2. Rastrear dependencias a largo plazo
3. Mantener el orden en las secuencias
4. Compartir parámetros a lo largo de la secuencia



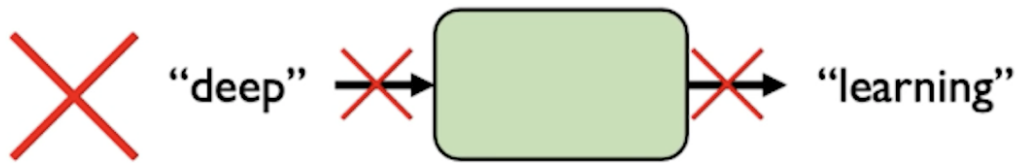
Un problema de modelado de secuencias: Predicción de la siguiente palabra

“This morning I took my cat for a walk.”

given these words

predict the
next word

Representing Language to a Neural Network

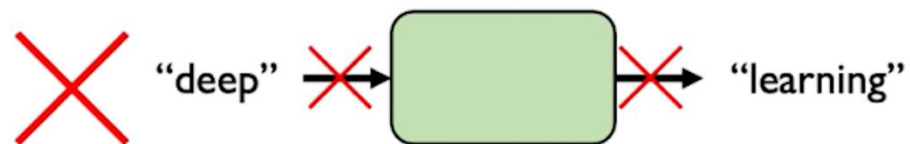


Neural networks cannot interpret words



Neural networks require numerical inputs

Codificando Lenguaje Natural en una RN



Neural networks cannot interpret words



Neural networks require numerical inputs

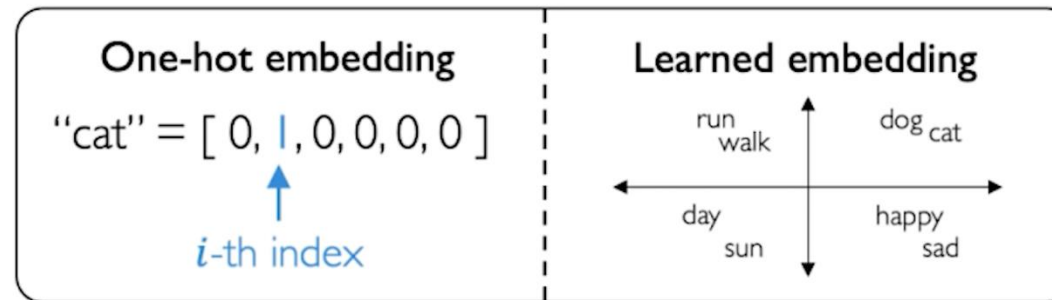
Embedding: transform indexes into a vector of fixed size.

this cat for
my took
a | walk
morning

1. Vocabulary:
Corpus of words

a → 1
cat → 2
... → ...
walk → N

2. Indexing:
Word to index



3. Embedding:
Index to fixed-sized vector

Codificando Lenguaje Natural en una RN

La comida fue muy buena

vs.

Visitamos un restaurante durante la comida

vs.

Esta tarde visitaré al médico debido a que tengo algunos síntomas de gripe

Codificando Lenguaje Natural en una RNN

Diferencias entre secuencias



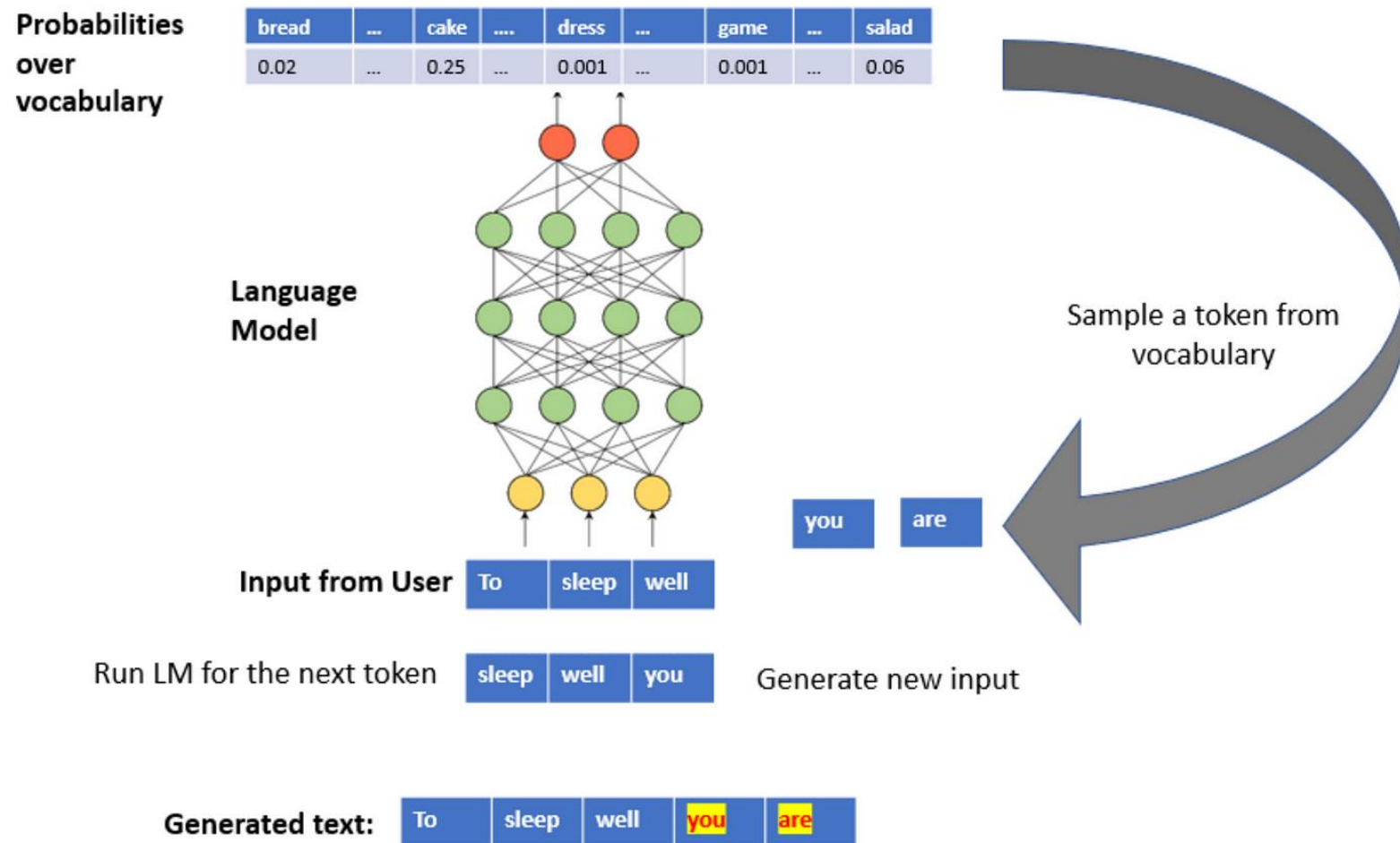
The food was good, not bad at all.

vs.

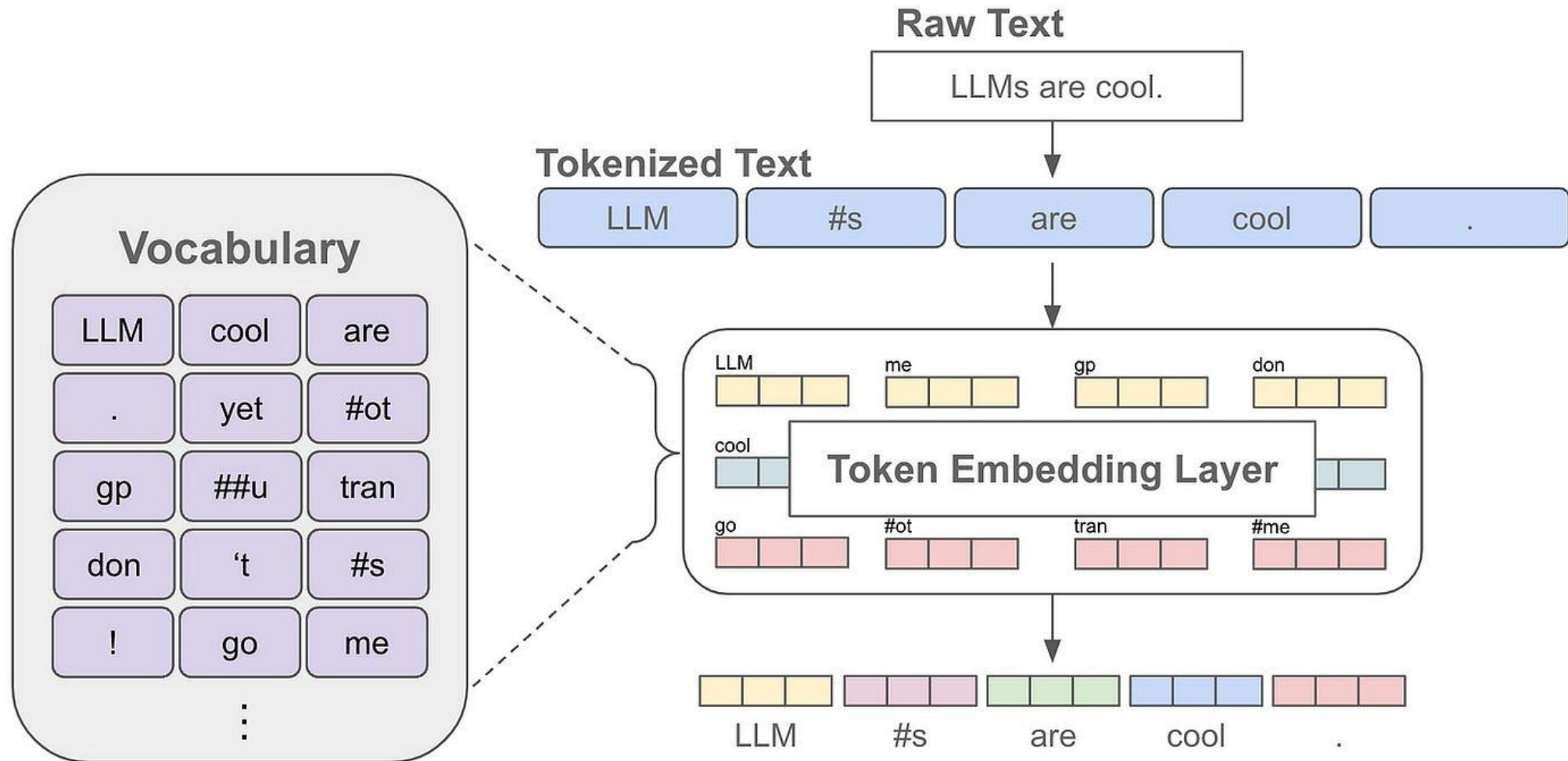
The food was bad, not good at all.



Ejemplo: Predicción de la siguiente palabra



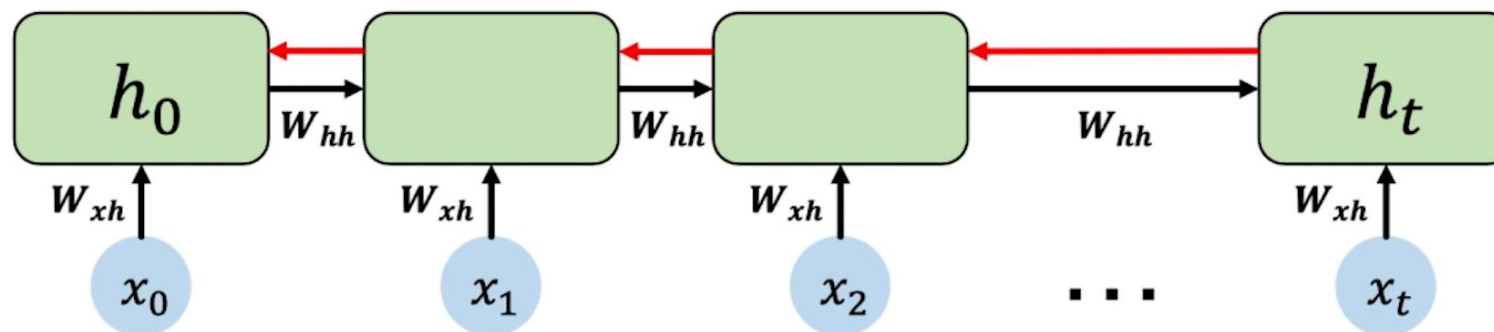
Ejemplo: Predicción de la siguiente palabra





Back Propagation A través del tiempo

Backpropagation: a través del tiempo



Computing the gradient wrt h_0 involves **many factors of w_{hh}** + **repeated gradient computation!**

Many values > 1 :
exploding gradients

Gradient clipping to
scale big gradients

Many values < 1 :
vanishing gradients

1. Activation function
2. Weight initialization
3. Network architecture

El problema de las dependencias a largo plazo

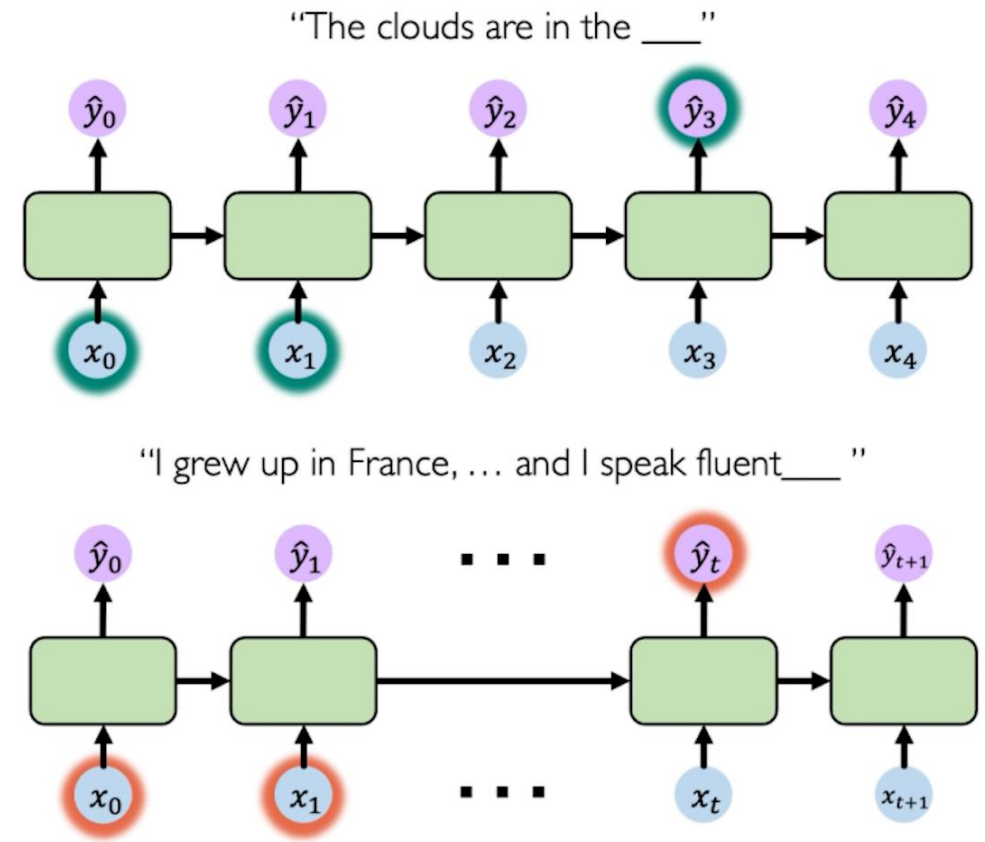
¿Porqué las dependencias a largo plazo son un problema?



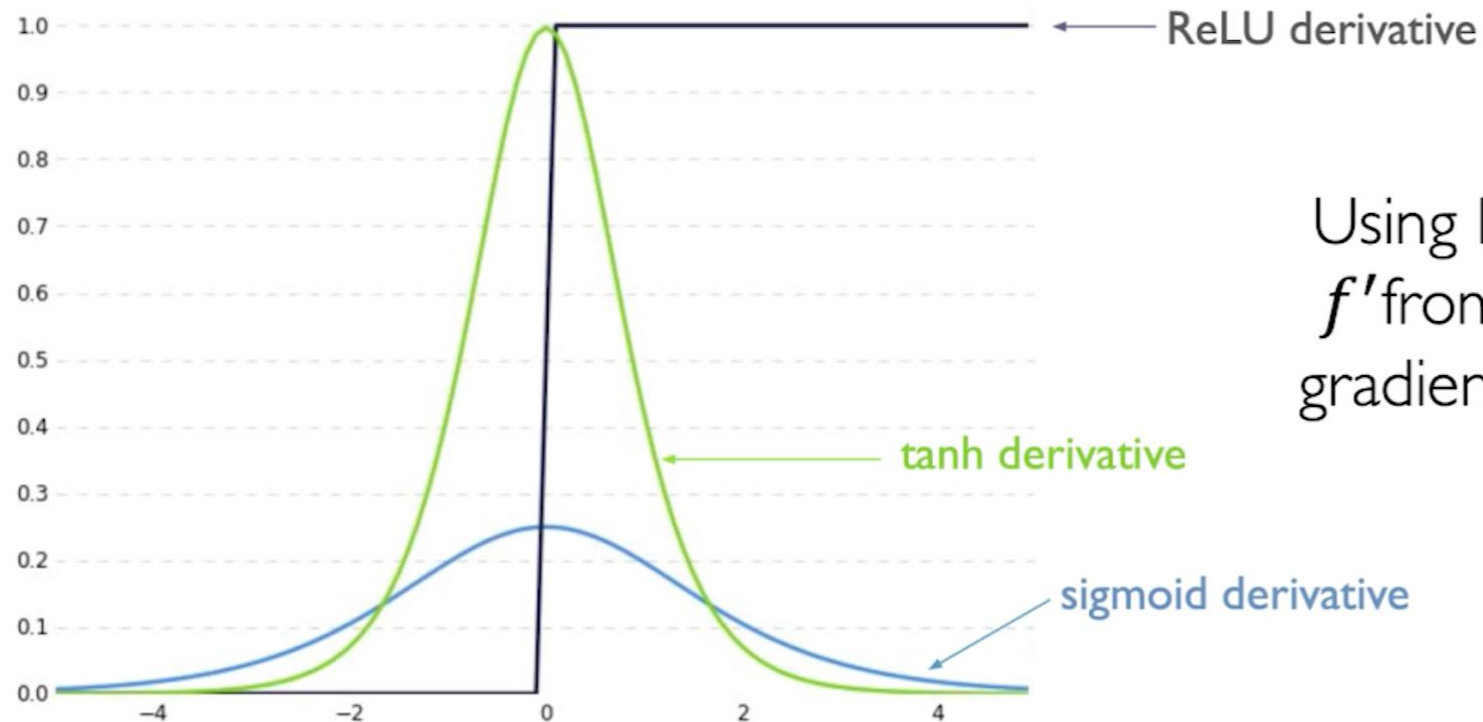
Multiplican múltiples números pequeños



Esta propagación genera gradientes cada vez más pequeños



Posible solución 1: Funciones de activación



Using ReLU prevents f' from shrinking the gradients when $x > 0$

Posible solución 2:

Inicialización de parámetros

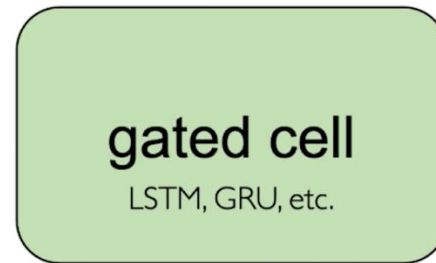
- Inicializar pesos usando una matriz identidad
 - Inicializar bias con cero
- Ayuda a prevenir que los pesos se **desvanezcan**.

$$I_n = \begin{pmatrix} 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 \end{pmatrix}$$

Posible solución 3:

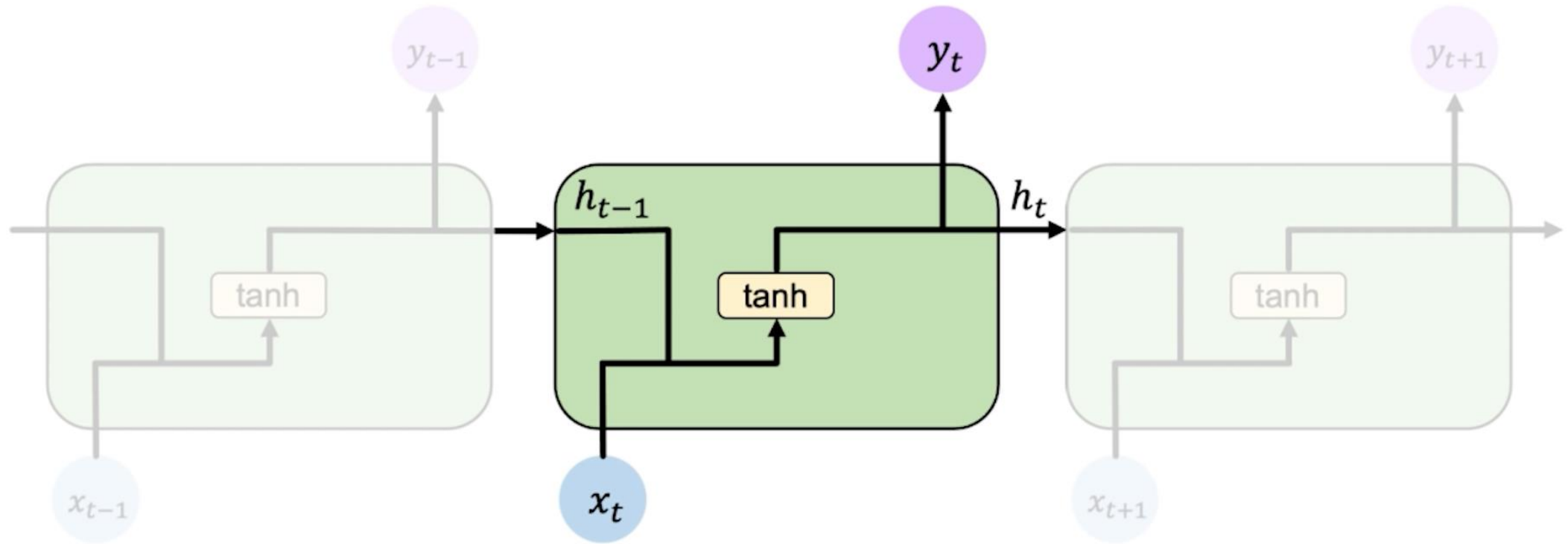
Celdas con compuertas

- Utilizar unidades recurrentes con compuertas para controlar que información fluye por la celda.

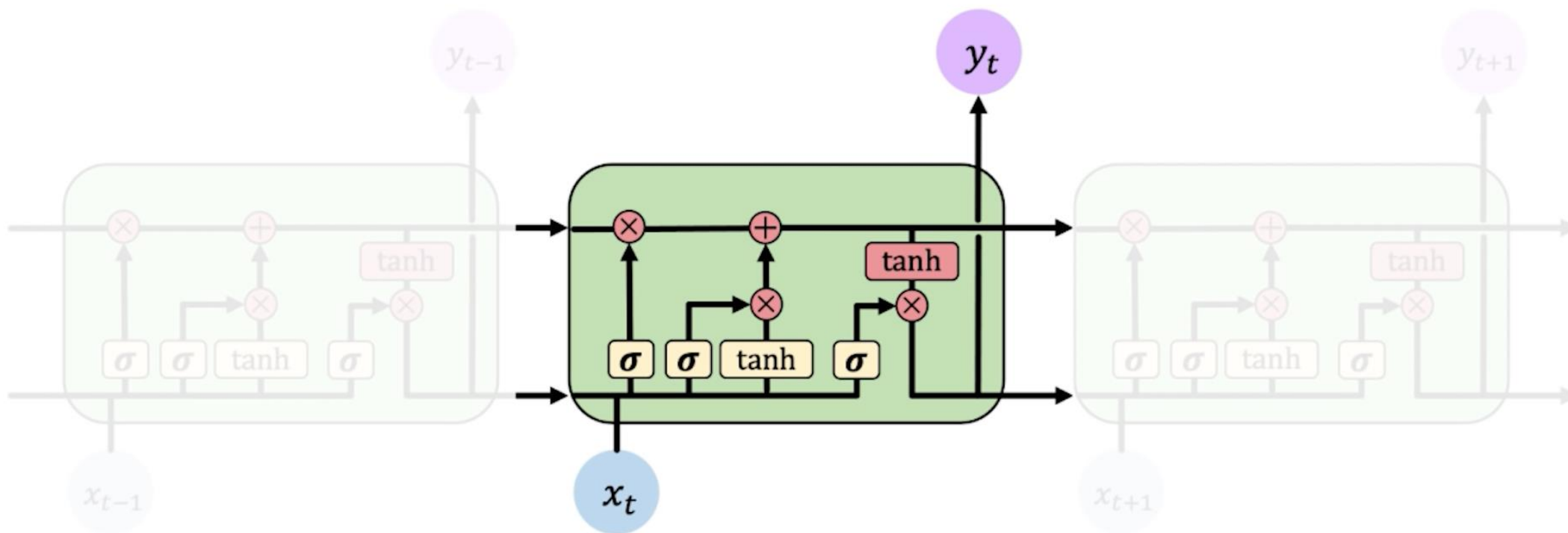


- Memoria de corto y largo plazo (LSTMs), las redes se basan e celdas con compuertas para registrar información a lo largo de múltiples pasos.

Redes Neuronales Recurrentes Estándar



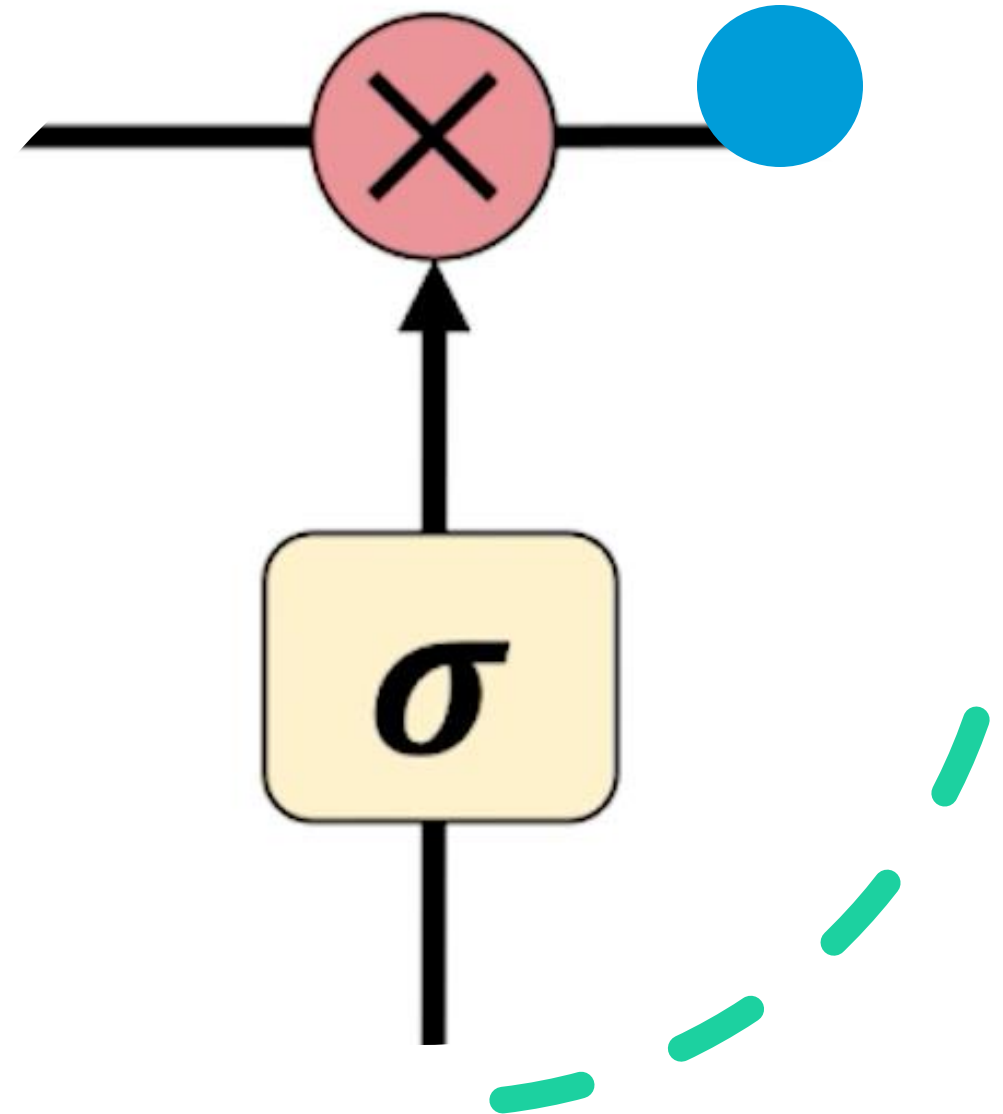
Long Short Term Memory (LSTMs)



```
tf.keras.layers.LSTM(num_units)
```

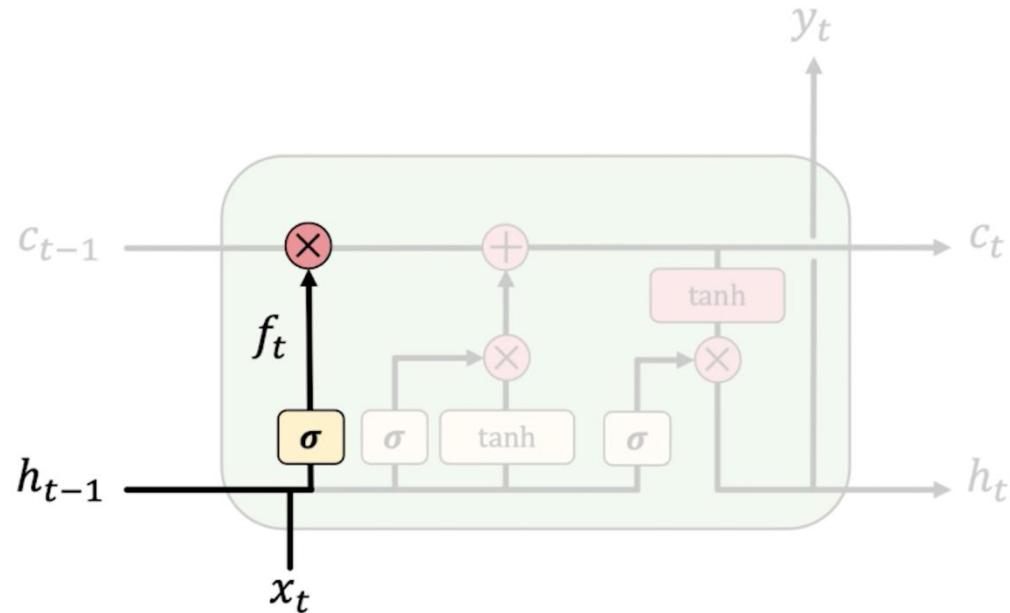
Long Short Term Memory (LSTMs)

- La información se agregar o elimina a través de estructuras llamadas compuertas.
- Las compuertas dejan fluir la información, este se puede lograr a través de una función sigmoide.



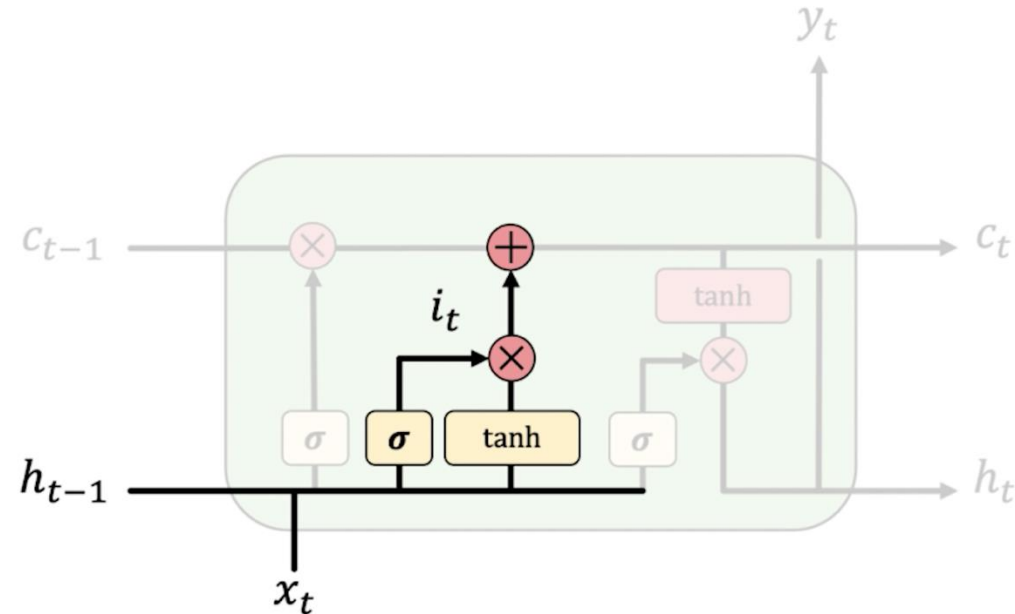
Long Short Term Memory (LSTMs)

1) **Forget** 2) Store 3) Update 4) Output
LSTMs **forget irrelevant** parts of the previous state



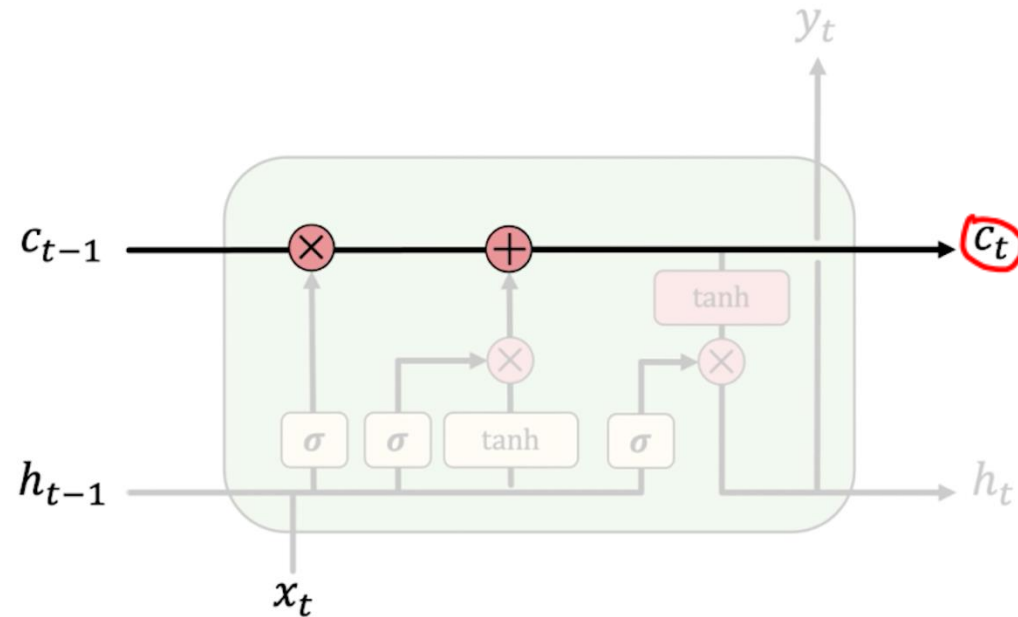
Long Short Term Memory (LSTMs)

1) Forget **2) Store** 3) Update 4) Output
LSTMs **store relevant** new information into the cell state



Long Short Term Memory (LSTMs)

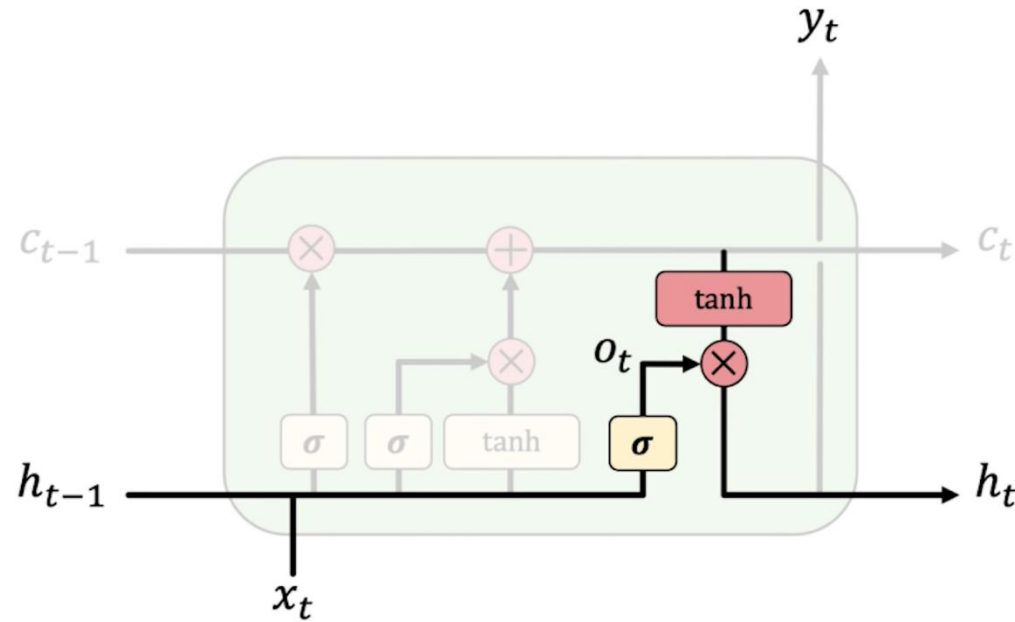
1) Forget 2) Store **3) Update** 4) Output
LSTMs **selectively update** cell state values



Long Short Term Memory (LSTMs)

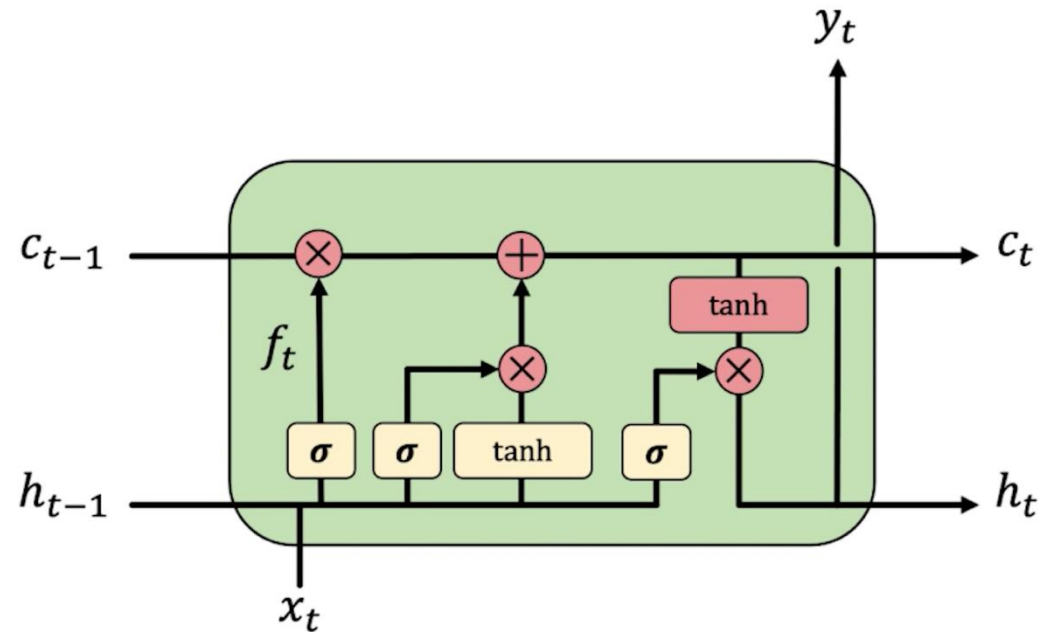
1) Forget 2) Store 3) Update 4) **Output**

The **output gate** controls what information is sent to the next time step



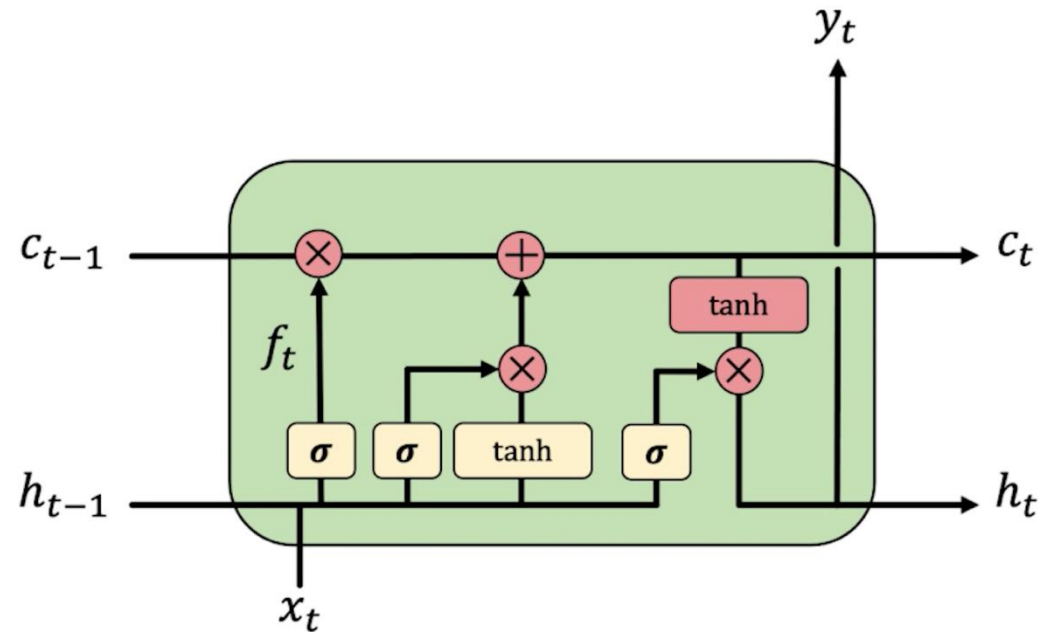
Long Short Term Memory (LSTMs)

1) Forget 2) Store 3) Update 4) Output

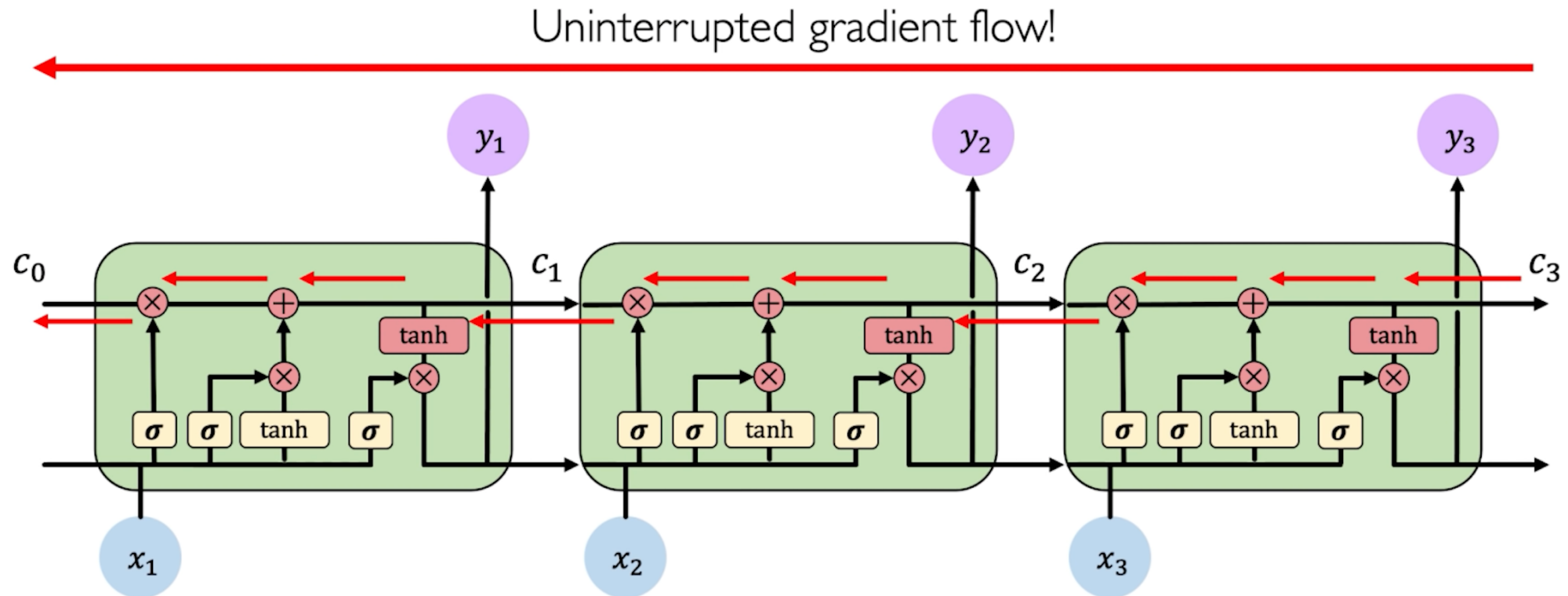


Long Short Term Memory (LSTMs)

1) Forget 2) Store 3) Update 4) Output



Long Short Term Memory (LSTMs)



LSTMs: Conceptos principales

1. Mantienen un estado separado de las salidas
2. Utiliza compuertas para controlar el flujo de información
 - Las compuertas eliminan información irrelevante
 - Almacenan información relevante de la entrada actual
 - Actualizan de forma selectiva el estado de una celda
 - Las compuertas de salida retornan una versión filtrada del estado de la celda.
3. Backpropagation con uninterrupted gradient flow



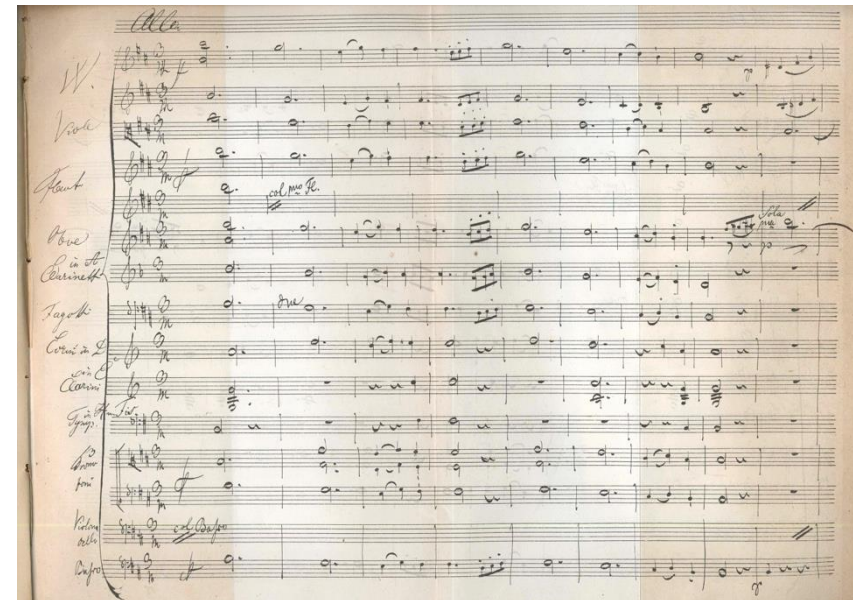
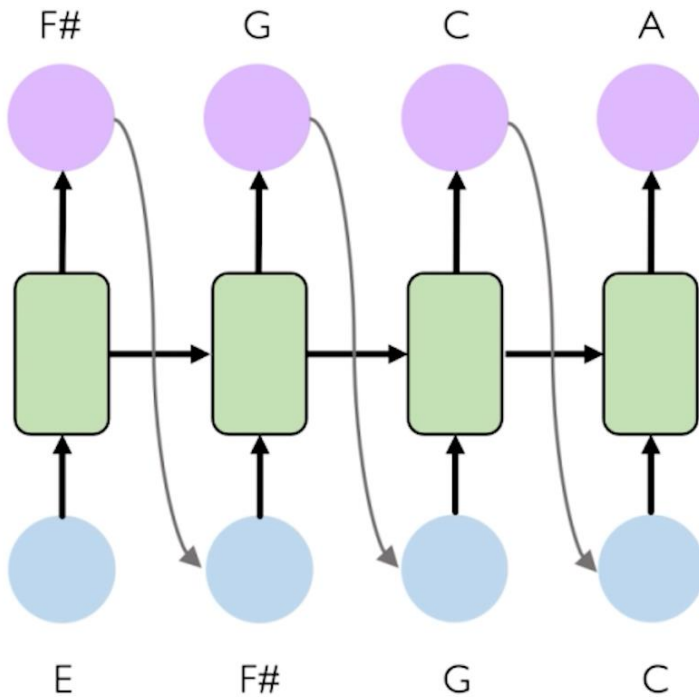
Aplicaciones de RNN

Ejemplo: Generación de música



Sinfonía Incompleta de Schubert

- **Entrada:** Conjunto de notas musicales
- **Salida:** Siguiente nota musical



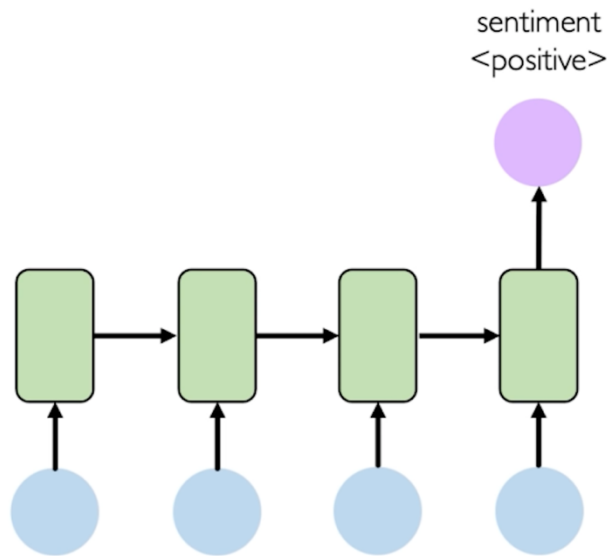
https://www.youtube.com/watch?v=RCo8Feho1RI&ab_channel=MusicalSeries

Listening to
1st movement

Sinfonía incompleta Schubert



Ejemplo: Clasificación de sentimientos



- **Entrada:** Secuencia de palabras
- **Salida:** Probabilidad de obtener un sentimiento positivo.

Problema de clasificación

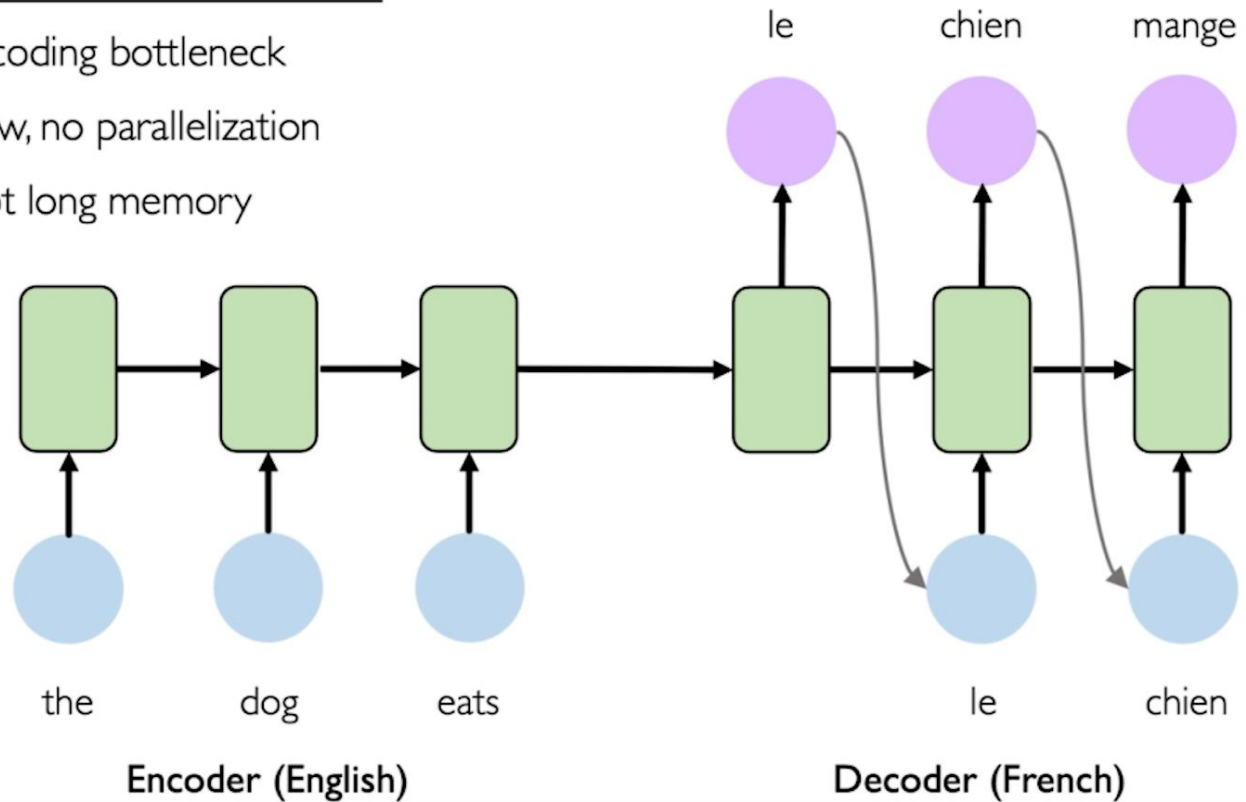


```
loss = tf.nn.softmax_cross_entropy_with_logits(y, predicted)
```

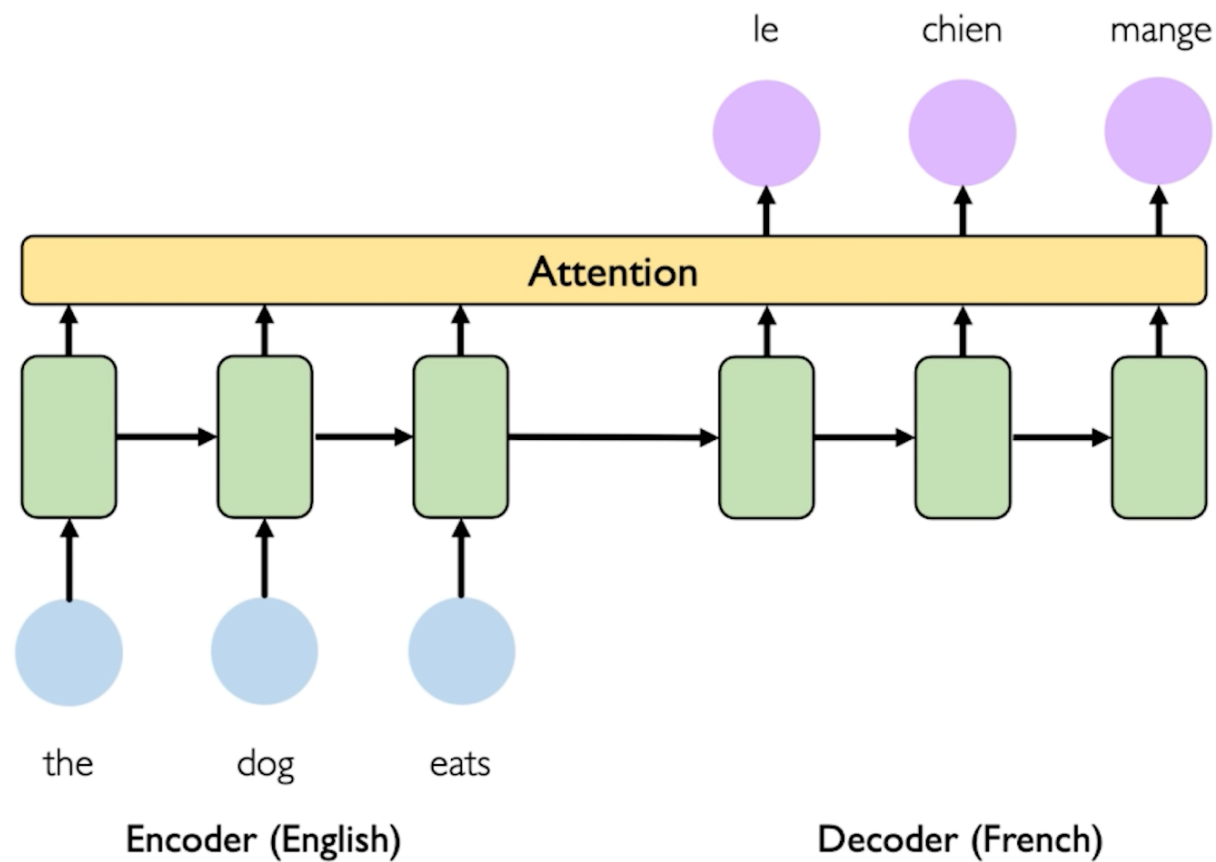
Traducción automática

Potential Issues

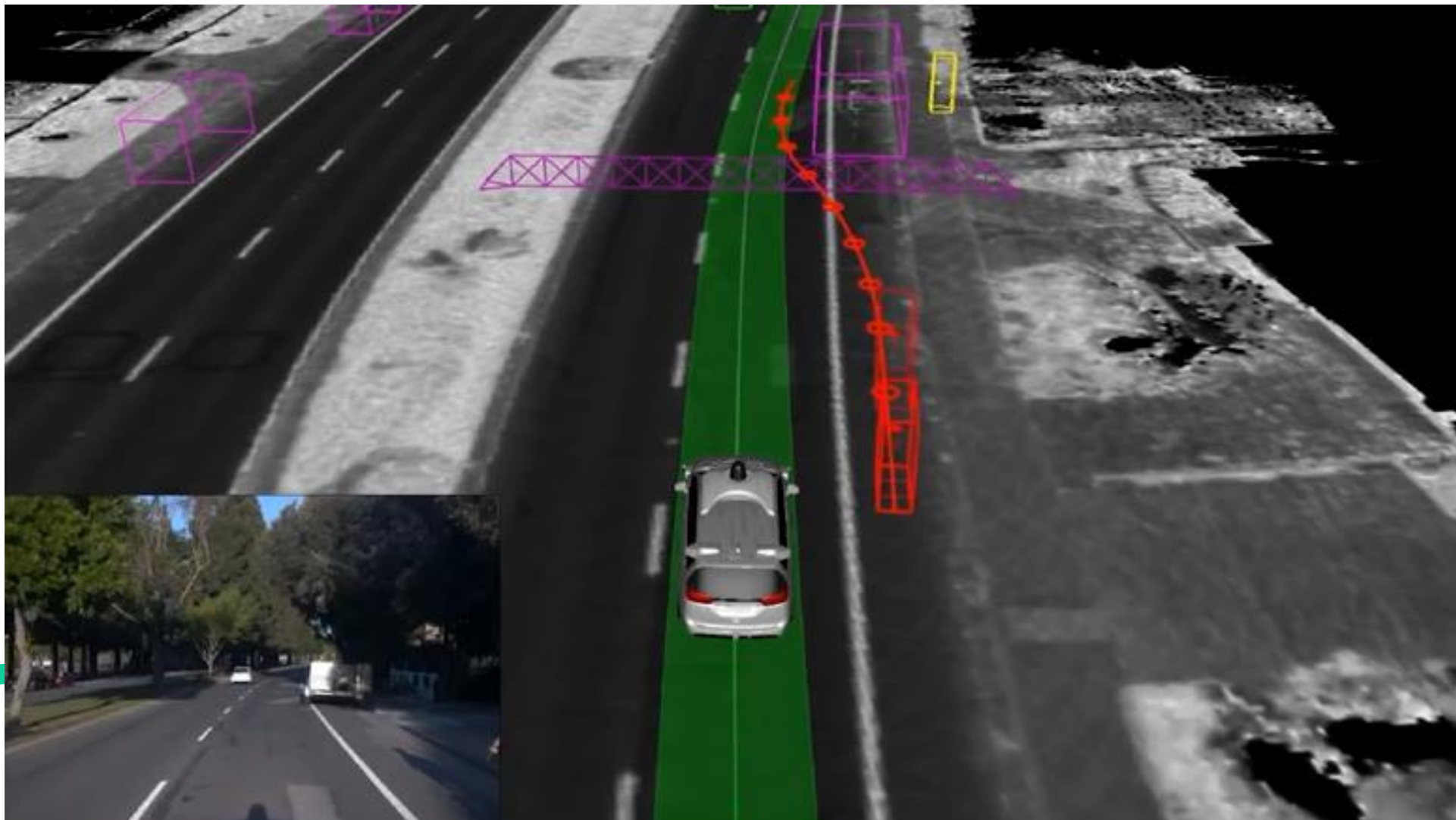
-  Encoding bottleneck
-  Slow, no parallelization
-  Not long memory



Traducción automática



Predicción de trayectorias



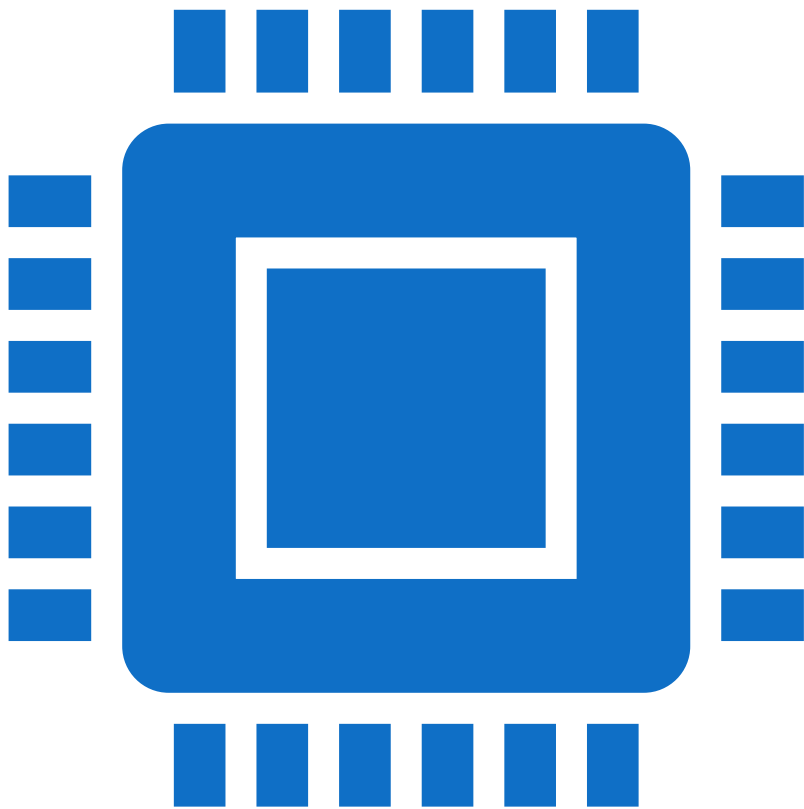
Procesamiento del Lenguaje Natural (PLN)

1. **Análisis de Sentimiento:** Las RNN pueden analizar el sentimiento de las frases teniendo en cuenta la secuencia de palabras.
2. **Traducción Automática:** Pueden traducir frases de un idioma a otro.
3. **Reconocimiento de Entidades Nombradas:** Las RNN pueden identificar entidades como nombres, lugares y organizaciones en texto.
4. **Reconocimiento de Voz:** Pueden convertir el lenguaje hablado en texto.

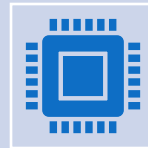


Procesamiento de Señales

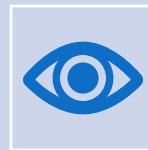
- 1. Síntesis de Audio:** Pueden generar música u otras formas de audio basándose en muestras previas.
- 2. Descripción de Imágenes:** Las RNN se pueden combinar con redes neuronales convolucionales (CNN) para generar descripciones textuales de imágenes.



Robótica



Planificación de Rutas: Las RNN pueden analizar una secuencia de datos de sensores para ayudar en la navegación de robots.



Reconocimiento de Gestos: Pueden reconocer e interpretar gestos humanos para una interacción humano-máquina más natural.



Atención Médica

1. Monitoreo de Pacientes: Las RNN pueden analizar datos de series temporales de sistemas de monitoreo de pacientes para proporcionar señales de alerta temprana.

2. Diagnóstico Médico: Pueden interpretar secuencias de imágenes médicas para ayudar en el diagnóstico.



Finanzas

- 1.Comercio Algorítmico:** Las RNN pueden emplearse para tomar decisiones de compra o venta basadas en una secuencia de indicadores del mercado financiero.
- 2.Evaluación de Crédito:** Pueden evaluar el riesgo de prestar a individuos o entidades analizando datos financieros históricos.

Juegos y Entretenimiento



IA en Juegos: Las RNN pueden modelar el comportamiento de los jugadores o entidades del juego a lo largo del tiempo.



Recomendación de Contenido: Pueden analizar la secuencia de interacciones del usuario para recomendar contenido relevante.

Otros

01

Vehículos Autónomos:

Las RNN pueden procesar secuencias de datos de sensores para mejorar la navegación y toma de decisiones.

02

Optimización de la Cadena de Suministro:

Pueden analizar datos históricos para mejorar la gestión de inventario.

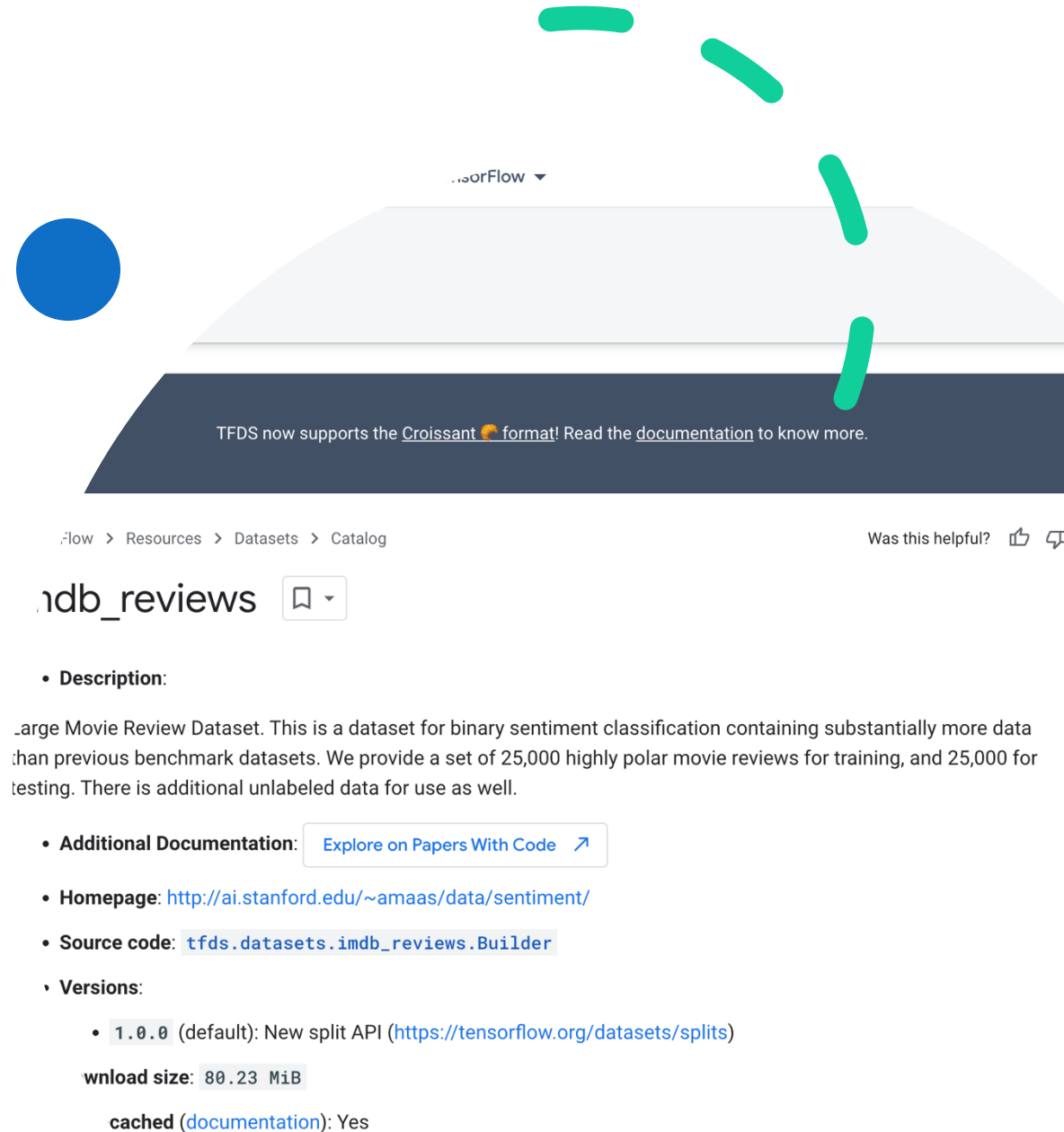
The IMDb logo is a large, yellow, rounded rectangle with the letters "IMDb" in a bold, black, sans-serif font. It is centered in the middle of the image, overlapping a background of movie posters.

IMDb

Internet Movie Database

Exploración del set de datos

https://www.tensorflow.org/datasets/catalog/imdb_reviews



The screenshot shows the TensorFlow Datasets (TFDS) catalog page for the 'imdb_reviews' dataset. The page has a dark blue header with the TensorFlow logo and a navigation bar. The main content area is white and contains the following information:

- Dataset Name:** `imdb_reviews`
- Description:** Large Movie Review Dataset. This is a dataset for binary sentiment classification containing substantially more data than previous benchmark datasets. We provide a set of 25,000 highly polar movie reviews for training, and 25,000 for testing. There is additional unlabeled data for use as well.
- Additional Documentation:** [Explore on Papers With Code](#)
- Homepage:** <http://ai.stanford.edu/~amaas/data/sentiment/>
- Source code:** `tfds.datasets.imdb_reviews.Builder`
- Versions:**
 - 1.0.0** (default): New split API (<https://tensorflow.org/datasets/splits>)
- Download size:** 80.23 MiB
- cached** ([documentation](#)): Yes

IMDb

Positivos



- Very smart, sometimes shocking, I just love it. It shoved one more side of David's brilliant talent. He impressed me greatly! David is the best. The movie captivates your attention for every second.
- The little girl Desi is so adorable... I cant think of a more beautiful story then this one here. It will make you cry, laugh, and believe. ...

Negativos



- Bad acting, bad lighting, bad plot!! This had the quality of a porno movie. I have seen more interesting home video of a boring wedding than this movie...it sucked big time!! ...
- Were I not with friends, and so cheap, I would have walked out. It failed miserably as satire and didn't even have the redemption of camp.

Código en Colab



Importar Librerías

✓ Paso 1: Importación de Bibliotecas Necesarias

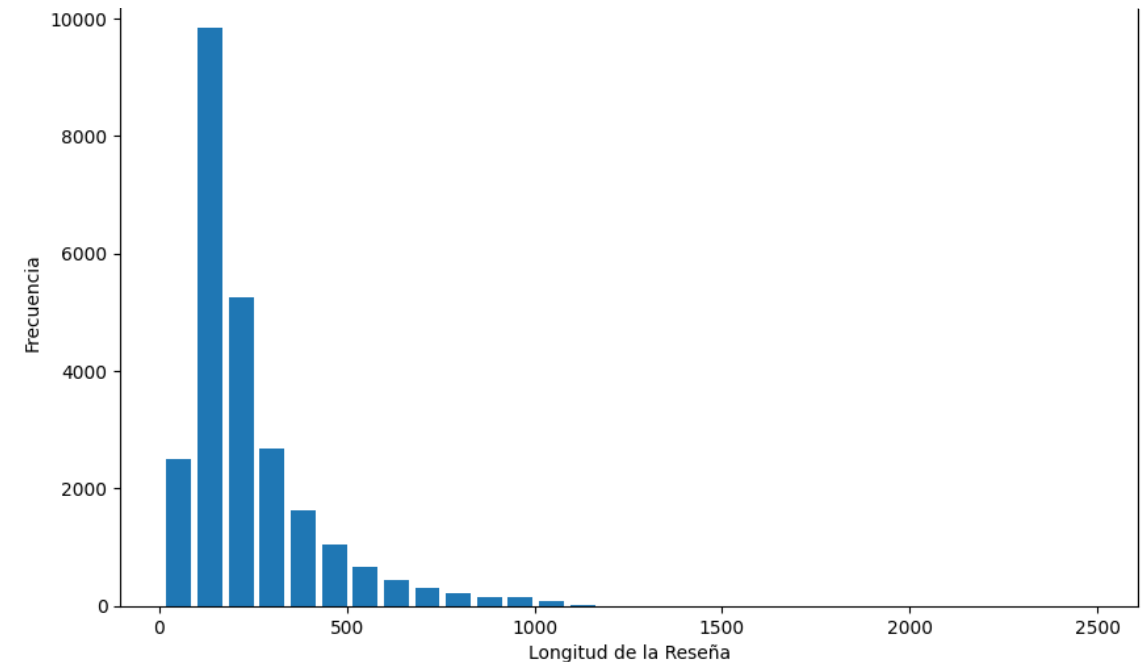
✓
0s

```
[9] 1  # Import necessary libraries
    2  import numpy as np
    3  import tensorflow as tf
    4  from tensorflow.keras.models import Sequential
    5  from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout, Bidirectional
    6  from tensorflow.keras.datasets import imdb
    7  from tensorflow.keras.preprocessing.sequence import pad_sequences
    8  import matplotlib.pyplot as plt
    9  from sklearn.metrics import confusion_matrix
   10  import seaborn as sns
   11  import ipywidgets as widgets
   12  from IPython.display import display
```

Carga de datos

✓ Paso 2: Carga del Conjunto de Datos de IMDB

```
[10] 1
      2 # Load the IMDB dataset
      3 vocabulary_size = 10000
      4 (x_train, y_train), (x_test, y_test) = imdb.load_data(num_
      5
      6 # Visualización de la longitud de las reseñas
      7 lengths = [len(x) for x in x_train]
      8 plt.figure(figsize=(10,6))
      9 plt.hist(lengths, bins=30, rwidth=0.8)
     10 plt.title('Distribución de la Longitud de las Reseñas')
     11 plt.xlabel('Longitud de la Reseña')
     12 plt.ylabel('Frecuencia')
     13 plt.show()
```



Visualización de datos

✓ Paso 3: Visualización de los Datos Iniciales

```
[11] 1
      2 # Obtener el índice de palabras de IMDB
      3 word_index = imdb.get_word_index()
      4 reverse_word_index = dict([(value, key) for (key, value) in word_index.items()])
      5
      6 def print_review(index=0):
      7     # Decodificar la reseña seleccionada
      8     decoded_review = ' '.join([reverse_word_index.get(i - 3, '?') for i in x_train[index]])
      9
     10     # Imprimir la reseña codificada y decodificada
     11     print("Etiqueta: ", "positive" if (y_train[index] == 1) else "negative", "\n")
     12     print("Reseña codificada:", x_train[index])
     13     print("\nReseña decodificada:\n", decoded_review)
     14
     15     # Crear un control deslizante para seleccionar el ID de la reseña
     16     slider = widgets.IntSlider(value=0, min=0, max=len(x_train)-1, step=1, description='ID de Reseña:')
     17     widgets.interactive(print_review, index=slider)
```

```
0
positive

codificada: [1, 14, 22, 16, 43, 530, 973, 1622, 1385, 65, 458, 4468, 66, 3941, 4, 173, 36, 256, 5,
50, 670, 2, 9, 35, 480, 284, 5, 150, 4, 172, 112, 167, 2, 336, 385, 39, 4, 172, 4536, 1111, 17,
92, 50, 16, 6, 147, 2025, 19, 14, 22, 4, 1920, 4613, 469, 4, 22, 71, 87, 12, 16, 43, 530, 38, 76
22, 17, 515, 17, 12, 16, 626, 18, 2, 5, 62, 386, 12, 8, 316, 8, 106, 5, 4, 2223, 5244, 16, 480,
2, 16, 38, 619, 5, 25, 124, 51, 36, 135, 48, 25, 1415, 33, 6, 22, 12, 215, 28, 77, 52, 5, 14, 40
, 117, 5952, 15, 256, 4, 2, 7, 3766, 5, 723, 36, 71, 43, 530, 476, 26, 400, 317, 46, 7, 4, 2, 10
1, 15, 297, 98, 32, 2071, 56, 26, 141, 6, 194, 7486, 18, 4, 226, 22, 21, 134, 476, 26, 480, 5, 1
6, 28, 224, 92, 25, 104, 4, 226, 65, 16, 38, 1334, 88, 12, 16, 283, 5, 16, 4472, 113, 103, 32, 1
32]

codificada:
ilm was just brilliant casting location scenery story direction everyone's really suited the par
ould just imagine being there robert ? is an amazing actor and now the same being director ? fat
scottish island as myself so i loved the fact there was a real connection with this film the wit
t the film were great it was just brilliant so much that i bought the film as soon as it was rel
recommend it to everyone to watch and the fly fishing was amazing really cried at the end it wa
what they say if you cry at a film it must have been good and this definitely was also ? to the
t played the ? of norman and paul they were just brilliant children are often left out of the
he stars that play them all grown up are such a big profile for the whole film but these childre
d be praised for what they have done don't you think the whole story was so lovely because it wa
life after all that was shared with us all
```

Visualización de datos

Preprocesamiento

(Estructurar datos en secuencias)

✓ Paso 4: Preprocesamiento de los Datos

```
[12] 1  # Preprocess the data
      2  max_length = 500
      3  x_train = pad_sequences(x_train, maxlen=max_length, padding='post')
      4  x_test = pad_sequences(x_test, maxlen=max_length, padding='post')
```

Construcción del modelo

▼ Paso 5: Construcción del Modelo de Clasificación con RNN

```
[13] 1 # Build the RNN classification model
      2 model = Sequential([
      3     Embedding(input_dim=vocabulary_size, output_dim=32, input_length=max_length),
      4     LSTM(32),
      5     Dense(1, activation='sigmoid')
      6 ])
      7
      8 model.summary()
      9
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 500, 32)	320000
lstm_1 (LSTM)	(None, 32)	8320
dense_1 (Dense)	(None, 1)	33
Total params: 328353 (1.25 MB)		
Trainable params: 328353 (1.25 MB)		
Non-trainable params: 0 (0.00 Byte)		

Entrenamiento del modelo

✓ Paso 6: Entrenamiento del Modelo

```
[14] 1
      2 # Train the model
      3 model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
      4 history = model.fit(x_train, y_train, epochs=10, validation_data=(x_test, y_test), batch_size=128)
      5
```

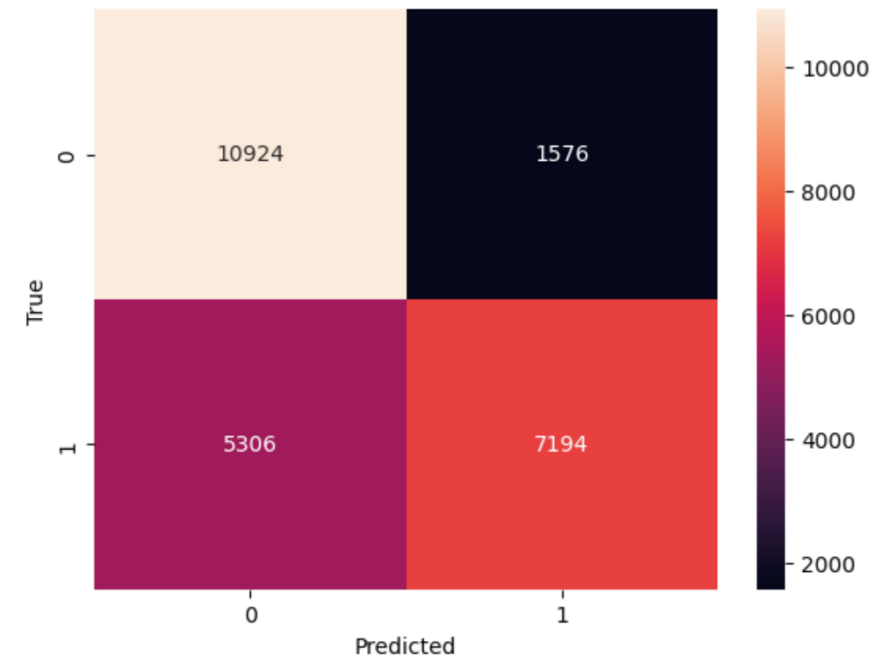
```
Epoch 1/10
196/196 [=====] - 27s 126ms/step - loss: 0.6928 - accuracy: 0.5095 - val_loss: 0.6924 - val_accuracy: 0.5071
Epoch 2/10
196/196 [=====] - 19s 98ms/step - loss: 0.6889 - accuracy: 0.5071 - val_loss: 0.6872 - val_accuracy: 0.5158
Epoch 3/10
196/196 [=====] - 16s 82ms/step - loss: 0.6749 - accuracy: 0.5325 - val_loss: 0.6834 - val_accuracy: 0.5152
Epoch 4/10
196/196 [=====] - 14s 70ms/step - loss: 0.6199 - accuracy: 0.6682 - val_loss: 0.7795 - val_accuracy: 0.5325
Epoch 5/10
196/196 [=====] - 15s 75ms/step - loss: 0.6303 - accuracy: 0.6282 - val_loss: 0.6896 - val_accuracy: 0.5072
Epoch 6/10
196/196 [=====] - 9s 48ms/step - loss: 0.6748 - accuracy: 0.5355 - val_loss: 0.6861 - val_accuracy: 0.5173
Epoch 7/10
196/196 [=====] - 9s 45ms/step - loss: 0.6583 - accuracy: 0.5435 - val_loss: 0.6892 - val_accuracy: 0.5251
Epoch 8/10
196/196 [=====] - 8s 42ms/step - loss: 0.6199 - accuracy: 0.5969 - val_loss: 0.6034 - val_accuracy: 0.6880
Epoch 9/10
196/196 [=====] - 7s 38ms/step - loss: 0.5394 - accuracy: 0.7345 - val_loss: 0.5875 - val_accuracy: 0.7245
Epoch 10/10
196/196 [=====] - 8s 42ms/step - loss: 0.5239 - accuracy: 0.7567 - val_loss: 0.5862 - val_accuracy: 0.7247
```

Evaluación del Modelo

✓ Paso 7: Evaluación del Modelo y Visualización de Resultados

```
[15] 1
      2 # Evaluate the model
      3 loss, accuracy = model.evaluate(x_test, y_test)
      4 print(f"Test Accuracy: {accuracy*100:.2f}%")
      5
      6 # Predictions
      7 predictions = (model.predict(x_test) > 0.5).astype("int32")
      8
      9 # Confusion Matrix
     10 cm = confusion_matrix(y_test, predictions)
     11 sns.heatmap(cm, annot=True, fmt="d")
     12 plt.xlabel('Predicted')
     13 plt.ylabel('True')
     14 plt.show()
     15
```

782/782 [=====] - 7s 9ms/step - loss: 0.5862 - accuracy: 0.7247
Test Accuracy: 72.47%
782/782 [=====] - 6s 8ms/step



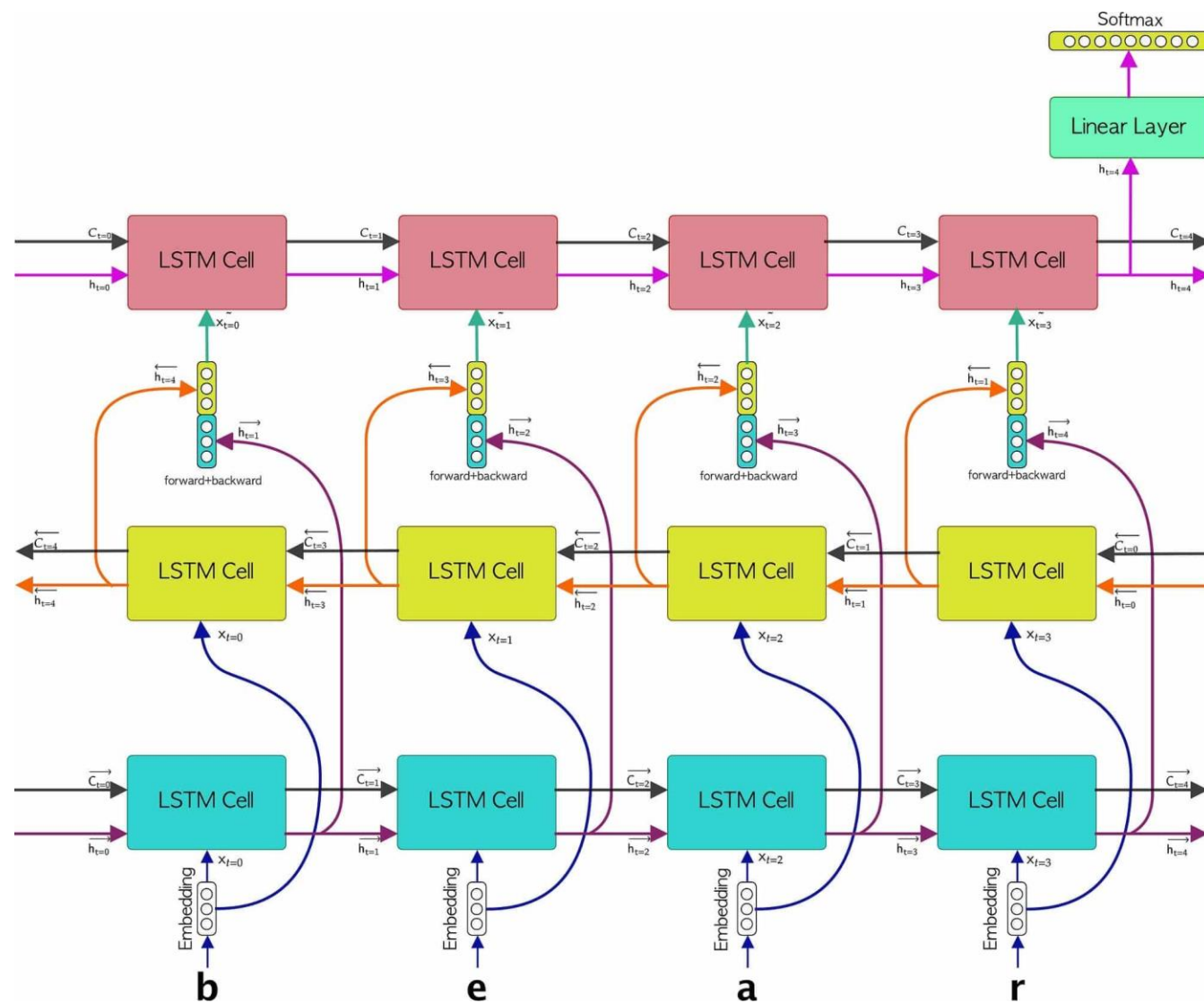
Ejercicios: Mejorar el rendimiento de la arquitectura

- Disminuir el tamaño de batch
- Incrementar el número de épocas
- Graficar Pérdida y Accuracy

¿Qué efecto tienen estos elementos?



LSTM Bidireccional



Ejercicio 2: Mejorando la arquitectura

```
# Parámetros del modelo
vocabulary_size = 10000
max_length = 500
embedding_dim = 32
lstm_units = 64
dropout_rate = 0.2
```

```
# Construcción del modelo con mejoras
model = Sequential([
    Embedding(input_dim=vocabulary_size, output_dim=embedding_dim, input_length=max_length),
    Bidirectional(LSTM(lstm_units, return_sequences=True)),
    Dropout(dropout_rate),
    Bidirectional(LSTM(lstm_units)),
    Dropout(dropout_rate),
    Dense(1, activation='sigmoid')
])
```



Ejercicios complementarios

- **Ejercicio 1: Análisis de Sentimientos en el Dataset yelp_polarity_reviews**
 - **Objetivo:** Construir un modelo para predecir la polaridad (positiva/negativa) de reseñas de restaurantes en Yelp.
 - **Tareas:**
 - Cargar el dataset yelp_polarity_reviews desde TFDS.
 - Preprocesar el texto y preparar los datos para el entrenamiento.
 - Diseñar y entrenar una RNN para clasificar las reseñas.
 - Optimizar el modelo mediante ajuste de hiperparámetros y evaluar su rendimiento.
- **Ejercicio 2: Clasificación de Noticias con el Dataset ag_news_subset**
 - **Objetivo:** Clasificar noticias en categorías como Mundo, Deportes, Negocios y Ciencia/Tecnología.
 - **Tareas:**
 - Cargar el dataset ag_news_subset desde TFDS.
 - Implementar técnicas de preprocesamiento de texto adecuadas.
 - Construir un modelo de clasificación utilizando RNNs.
 - Evaluar la precisión del modelo por categoría y discutir los resultados.

Ejercicios complementarios

- **Ejercicio 3: Detección de Spam en Mensajes de Texto con sms_spam**
 - **Objetivo:** Desarrollar un modelo para identificar mensajes de texto spam.
 - **Tareas:**
 - Cargar el dataset sms_spam desde TFDS.
 - Realizar preprocesamiento de los mensajes de texto.
 - Diseñar y entrenar un modelo de RNN para la detección de spam.
 - Evaluar el modelo usando métricas como precisión y pérdida.
- **Ejercicio 4: Clasificación de sentimientos en twitter**
 - **Tareas**
 - Se deberá usar el dataset "Sentiment140",
 - Contiene 1,600,000 tweets extraídos usando la API de Twitter.
 - Los tweets han sido anotados (0 = negativo, 2 = neutro, 4 = positivo)
 - Están codificados de manera que se pueda determinar fácilmente el sentimiento de cada tweet.